Министерство науки и высшего образования РФ

Федеральное государственное автономное образовательное учреждение высшего образования «Омский государственный технический университет»

Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЕТ

ЛАБОРАТОРНЫЙ ПРАКТИКУМ ПО ДИСЦИПЛИНЕ

«Машинное обучение и анализ данных»

Выполнила:

студентка группы ПИН-232

Кожевников Е.В.

_____

(дата, подпись)

Проверил:

к.т.н., доцент Малков О.Б.

_____

(дата, подпись)

Омск 2026

СОДЕРЖАНИЕ

Лабораторная работа №1. Работа с библиотекой Pandas 3
Лабораторная работа №2. Визуальный анализ данных. 12
Лабораторная работа №3. Бинарная классификация. 17
Лабораторная работа №4. Многомерная регрессия. 23
Лабораторная работа №5. Кластеризация. 30
Лабораторная работа №6. Решение задачи классификации методом решающих деревьев. 41
ЗАКЛЮЧЕНИЕ 48

Лабораторная работа №1. Работа с библиотекой Pandas

Весь процесс и результат лабораторной работы №1 представлены на рисунках 1-11.

Задание 1

1. Импортируйте необходимые библиотеки

```
import ...
```
[66]

2. Импортируем наборы данных из файлов auto1.csv и auto2.csv и поместим их в датафреймы auto1 и auto2

```
df1 = pd.read_csv('auto1.csv')
```
[67]

```
df2 = pd.read_csv('auto2.csv')
```
[68]

3. Выведите 10 первых строк каждого датафрейма

```
df1.head(10)
```
[69]

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car	Unnamed: 9	Unnamed: 10	Unnamed: 11	Unnamed: 12	Unnamed: 13
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu	NaN	NaN	NaN	NaN	NaN
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320	NaN	NaN	NaN	NaN	NaN
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite	NaN	NaN	NaN	NaN	NaN
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst	NaN	NaN	NaN	NaN	NaN
4	17.0	8	302	140	3449	10.5	70	1	ford torino	NaN	NaN	NaN	NaN	NaN
5	15.0	8	429	198	4341	10.0	70	1	ford galaxie 500	NaN	NaN	NaN	NaN	NaN
6	14.0	8	454	220	4354	9.0	70	1	chevrolet impala	NaN	NaN	NaN	NaN	NaN
7	14.0	8	440	215	4312	8.5	70	1	plymouth fury iii	NaN	NaN	NaN	NaN	NaN
8	14.0	8	455	225	4425	10.0	70	1	pontiac catalina	NaN	NaN	NaN	NaN	NaN
9	15.0	8	390	190	3850	8.5	70	1	amc ambassador dpl	NaN	NaN	NaN	NaN	NaN

Рисунок 1 – Лабораторная работа №1

```
df2.head(10)
```
[70]

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car
0	33.0	4	91	53	1795	17.4	76	3	honda civic
1	20.0	6	225	100	3651	17.7	76	1	dodge aspen se
2	18.0	6	250	78	3574	21.0	76	1	ford granada ghia
3	18.5	6	250	110	3645	16.2	76	1	pontiac ventura sj
4	17.5	6	258	95	3193	17.8	76	1	amc pacer d/l
5	29.5	4	97	71	1825	12.2	76	2	volkswagen rabbit
6	32.0	4	85	70	1990	17.0	76	3	datsun b-210
7	28.0	4	97	75	2155	16.4	76	3	toyota corolla
8	26.5	4	140	72	2565	13.6	76	1	ford pinto
9	20.0	4	130	102	3150	15.7	76	2	volvo 245

4. Сколько строк и столбцов в каждом наборе данных?

```
rows, columns = df1.shape
print(rows)
print(columns)
```
[71]

```
198
14
```

```
rows, columns = df2.shape
print(rows)
print(columns)
```
[72]

```
200
9
```

5. Получите сводную информацию по датафрейму

```
df1.info()
```
[73]

```
RangeIndex: 198 entries, 0 to 197
```

Рисунок 2 – Лабораторная работа №1

5. Получите сводную информацию по датафрейму

```
df1.info()
[73]
```

```
RangeIndex: 198 entries, 0 to 197
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           198 non-null    float64
 1   cylinders     198 non-null    int64
 2   displacement  198 non-null    int64
 3   horsepower    198 non-null    object
 4   weight        198 non-null    int64
 5   acceleration  198 non-null    float64
 6   model         198 non-null    int64
 7   origin        198 non-null    int64
 8   car           198 non-null    object
 9   Unnamed: 9    0 non-null      float64
 10  Unnamed: 10   0 non-null      float64
 11  Unnamed: 11   0 non-null      float64
 12  Unnamed: 12   0 non-null      float64
 13  Unnamed: 13   0 non-null      float64
dtypes: float64(7), int64(5), object(2)
memory usage: 21.8+ KB
```

```
df2.info()
[74]
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           200 non-null    float64
 1   cylinders     200 non-null    int64
 2   displacement  200 non-null    int64
 3   horsepower    200 non-null    object
 4   weight        200 non-null    int64
 5   acceleration  200 non-null    float64
 6   model         200 non-null    int64
 7   origin        200 non-null    int64
 8   car           200 non-null    object
dtypes: float64(2), int64(5), object(2)
memory usage: 14.2+ KB
```

Рисунок 3 – Лабораторная работа №1

6. Что является индексом набора данных?

```
df1.index
[75]
```

```
RangeIndex(start=0, stop=198, step=1)
```

```
df2.index
[76]
```

```
RangeIndex(start=0, stop=200, step=1)
```

7. В наборе auto1 есть безымянные пустые столбцы, исправьте это

```
df1.head(5)
[77]
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car	Unnamed: 9	Unnamed: 10	Unnamed: 11	Unnamed: 12	Unnamed: 13
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu	NaN	NaN	NaN	NaN	NaN
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320	NaN	NaN	NaN	NaN	NaN
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite	NaN	NaN	NaN	NaN	NaN
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst	NaN	NaN	NaN	NaN	NaN
4	17.0	8	302	140	3449	10.5	70	1	ford torino	NaN	NaN	NaN	NaN	NaN

```
df1 = df1.drop(df1.columns[[-8, -4, -3, -2, -1]], axis=1)
[78]
```

```
df1.head(10)
[79]
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302	140	3449	10.5	70	1	ford torino
5	15.0	8	429	198	4341	10.0	70	1	ford galaxie 500

Рисунок 4 – Лабораторная работа №1

8. Объедините auto1 и auto2 в один датафрейм auto

```
auto = pd.concat([df1, df2], ignore_index=True)
```
[80]

```
auto.head(10)
```
[81]

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302	140	3449	10.5	70	1	ford torino
5	15.0	8	429	198	4341	10.0	70	1	ford galaxie 500
6	14.0	8	454	220	4354	9.0	70	1	chevrolet impala
7	14.0	8	440	215	4312	8.5	70	1	plymouth fury iii
8	14.0	8	455	225	4425	10.0	70	1	pontiac catalina
9	15.0	8	390	190	3850	8.5	70	1	amc ambassador dpl

```
auto.info()
```
[82]

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    int64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model         398 non-null    int64
 7   origin        398 non-null    int64
 8   car           398 non-null    object
dtypes: float64(2), int64(5), object(2)
memory usage: 28.1+ KB
```

Рисунок 5 – Лабораторная работа №1

9. В датафрейме auto отсутствует колонка owners (владельцы). Создайте

серию случайных чисел от 10 000 до 60 000 и добавьте столбец owners в датафрейм auto

```
owners_series = pd.Series(
    np.random.randint(10000, 60001, size=len(auto)),
    name='owners'
)

auto['owners'] = owners_series
```

[83]

```
auto.head(10)
```

[84]

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car	owners
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu	37536
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320	19708
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite	48931
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst	36801
4	17.0	8	302	140	3449	10.5	70	1	ford torino	12858
5	15.0	8	429	198	4341	10.0	70	1	ford galaxie 500	34066
6	14.0	8	454	220	4354	9.0	70	1	chevrolet impala	30390
7	14.0	8	440	215	4312	8.5	70	1	plymouth fury iii	55586
8	14.0	8	455	225	4425	10.0	70	1	pontiac catalina	11059
9	15.0	8	390	190	3850	8.5	70	1	amc ambassador dpl	47730

```
auto.info()
```

[85]

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    int64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
```

Рисунок 6 – Лабораторная работа №1

10. Каков временной интервал для рассматриваемых моделей?

```
min_date = auto['model'].min()
max_date = auto['model'].max()

print(f"Временной интервал: с {min_date} по {max_date}")
```

[86]

```
Временной интервал: с 70 по 82
```

11. Как в течение этого времени изменялось значение расхода топлива?

```
model70 = auto.groupby('model')['mpg'].mean()
model70
```

[87]

```
model
70    17.689655
71    21.250000
72    18.714286
73    17.100000
74    22.703704
75    20.266667
76    21.573529
77    23.375000
78    24.061111
79    25.093103
80    33.696552
81    30.334483
82    31.709677
Name: mpg, dtype: float64
```

12. Какие модели предпочитали покупатели (TOP-5)?

```
top5 = auto["car"].value_counts().head(10)
top5
```

[88]

```
car
ford pinto           6
ford maverick        5
amc matador          5
toyota corolla       5
amc hornet           4
peugeot 504          4
toyota corona        4
amc gremlin          4
chevrolet chevette   4
chevrolet impala     4
```

Рисунок 7 – Лабораторная работа №1

13. Связан ли расход топлива с количеством цилиндров?

```
auto.head(5)
[89]
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car	owners
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu	37536
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320	19708
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite	48931
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst	36801
4	17.0	8	302	140	3449	10.5	70	1	ford torino	12858

```
correlation = auto['mpg'].corr(auto['cylinders'])
correlation
[90]
```

```
np.float64(-0.7753962854205545)
```

```
cylinder_data = auto.groupby('cylinders')['mpg'].mean()

plt.figure(figsize=(10, 6))
bars = plt.bar(cylinder_data.index, cylinder_data.values,
               color='skyblue', edgecolor='black', width=0.7)

plt.title('Средний расход топлива по количеству цилиндров', fontsize=14, fontweight='bold')
plt.xlabel('Количество цилиндров', fontsize=12)
plt.ylabel('Средний расход топлива', fontsize=12)

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height + 0.1,
             f'{height:.1f}', ha='center', va='bottom', fontweight='bold')

plt.grid(axis='y', alpha=0.3)
plt.xticks(cylinder_data.index)
plt.tight_layout()
plt.show()
[91]
```

Рисунок 8 – Лабораторная работа №1

```
sns.lmplot(x="cylinders", y="mpg", data=auto)
plt.title('Средний расход топлива по количеству цилиндров')
[92]
```

```
Text(0.5, 1.0, 'Средний расход топлива по количеству цилиндров')
```

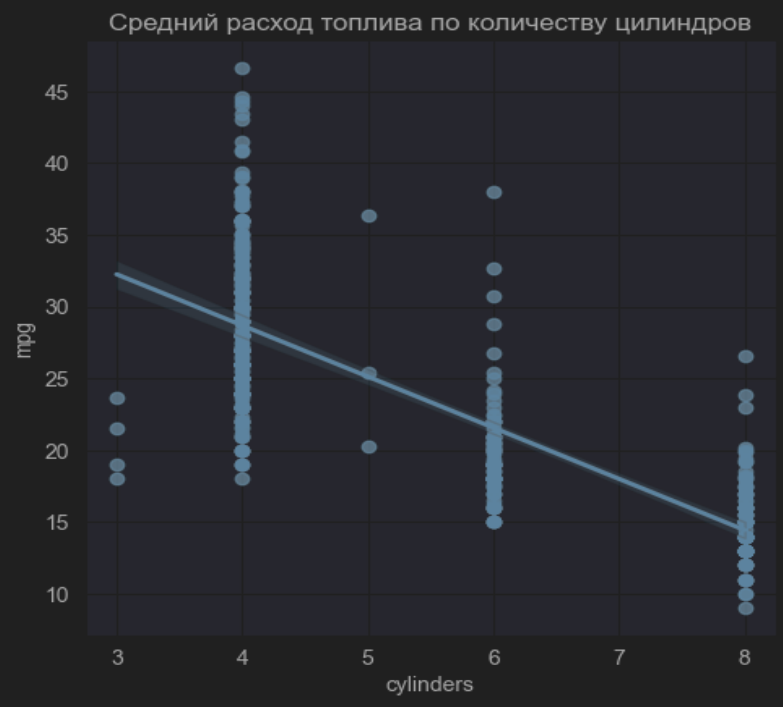


Рисунок 9 – Лабораторная работа №1

14. Докажите, что расход топлива зависит (или не зависит) от веса автомобиля.

```
auto.head(5)
```

[93]

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car	owners
0	18.0	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu	37536
1	15.0	8	350	165	3693	11.5	70	1	buick skylark 320	19708
2	18.0	8	318	150	3436	11.0	70	1	plymouth satellite	48931
3	16.0	8	304	150	3433	12.0	70	1	amc rebel sst	36801
4	17.0	8	302	140	3449	10.5	70	1	ford torino	12858

```
correlation = auto['mpg'].corr(auto['weight'])
correlation
```

[94]

```
np.float64(-0.8317409332443354)
```

```
sns.lmplot(x="weight", y="mpg", data=auto)
plt.title('Средний расход топлива от массы авто')
```

[95]

```
Text(0.5, 1.0, 'Средний расход топлива от массы авто')
```

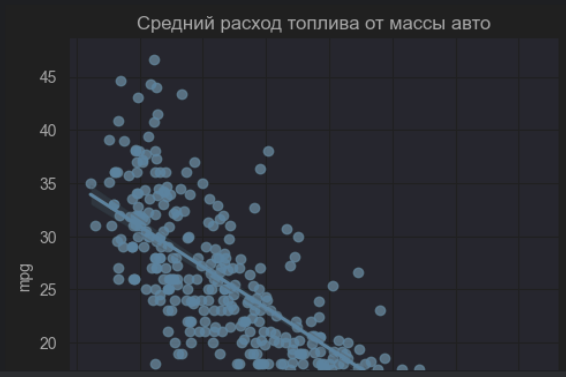


Рисунок 10 – Лабораторная работа №1

15. Отсортируйте датафрейм по полю weight (по убыванию)

```
auto_sorted = auto.sort_values(
    by=['weight'],
    ascending=False  # False для убывания, True для возрастания
)

auto_sorted
```

[96]

	mpg	cylinders	displacement	horsepower	weight	acceleration	model	origin	car	owners
44	13.0	8	400	175	5140	12.0	71	1	pontiac safari (sw)	11934
103	11.0	8	400	150	4997	14.0	73	1	chevrolet impala	12971
42	12.0	8	383	180	4955	11.5	71	1	dodge monaco (sw)	41510
90	12.0	8	429	198	4952	11.5	73	1	mercury marquis brougham	24876
95	12.0	8	455	225	4951	11.0	73	1	buick electra 225 custom	24490
...	...	...	...	...	...	...	...	...	...	...
53	31.0	4	71	65	1773	19.0	71	3	toyota corolla 1200	18470
345	35.1	4	81	60	1760	16.1	81	3	honda civic 1300	53516
343	39.1	4	79	58	1755	16.9	81	3	toyota starlet	42960
144	31.0	4	76	52	1649	16.5	74	3	toyota corona	26565
54	35.0	4	72	69	1613	18.0	71	3	datsun 1200	21026

398 rows × 10 columns

Рисунок 11 – Лабораторная работа №1

Лабораторная работа №2. Визуальный анализ данных.

Весь процесс и результат лабораторной работы №2 представлены на рисунках 11-19.

Задание 2

1. Импортируйте необходимые библиотеки

```
import ...
[1]
```

2. Импортируем набор данных из файла wine.csv и помещаем его в датафрейм df

```
df = pd.read_csv("wine.csv")
df.head(10)
[2]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	type
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	red
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	red
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	red
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	red
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	red
5	7.4	0.66	0.00	1.8	0.075	13.0	40.0	0.9978	3.51	0.56	9.4	5	red
6	7.9	0.60	0.06	1.6	0.069	15.0	59.0	0.9964	3.30	0.46	9.4	5	red
7	7.3	0.65	0.00	1.2	0.065	15.0	21.0	0.9946	3.39	0.47	10.0	7	red
8	7.8	0.58	0.02	2.0	0.073	9.0	18.0	0.9968	3.36	0.57	9.5	7	red
9	7.5	0.50	0.36	6.1	0.071	17.0	102.0	0.9978	3.35	0.80	10.5	5	red

3. Получить представлении о наборе данных с помощью методов shape, head, describe, info библиотеки pandas.

```
df.info(10)
[3]
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         6497 non-null   float64
 1   volatile acidity      6497 non-null   float64
 2   citric acid           6497 non-null   float64
```

Рисунок 11 – Лабораторная работа №2

```
df.describe()
[4]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000
mean	7.215307	0.339666	0.318633	5.443235	0.056034	30.525319	115.744574	0.994697	3.218501	0.531268	10.491801	5.818378
std	1.296434	0.164636	0.145318	4.757804	0.035034	17.749400	56.521855	0.002999	0.160787	0.148806	1.192712	0.873255
min	3.800000	0.080000	0.000000	0.600000	0.009000	1.000000	6.000000	0.987110	2.720000	0.220000	8.000000	3.000000
25%	6.400000	0.230000	0.250000	1.800000	0.038000	17.000000	77.000000	0.992340	3.110000	0.430000	9.500000	5.000000
50%	7.000000	0.290000	0.310000	3.000000	0.047000	29.000000	118.000000	0.994890	3.210000	0.510000	10.300000	6.000000
75%	7.700000	0.400000	0.390000	8.100000	0.065000	41.000000	156.000000	0.996990	3.320000	0.600000	11.300000	6.000000
max	15.900000	1.580000	1.660000	65.800000	0.611000	289.000000	440.000000	1.038980	4.010000	2.000000	14.900000	9.000000

```
rows, columns = df.shape
print(rows)
print(columns)
[5]
```

```
6497
13
```

```
df.isnull().sum()
[6]
```

```
fixed acidity         0
volatile acidity      0
citric acid           0
residual sugar        0
chlorides             0
free sulfur dioxide   0
total sulfur dioxide  0
density               0
pH                    0
sulphates             0
alcohol               0
quality               0
type                  0
dtype: int64
```

```
df.duplicated().sum()
[7]
```

```
np.int64(1177)
```

Рисунок 12 – Лабораторная работа №2

```python
df = df.drop_duplicates().reset_index(drop=True)
df.duplicated().sum()
```
[8]

```
np.int64(0)
```

4. Гистограммы и распределения

```python
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
if 'quality' in numeric_cols:
    numeric_cols.remove('quality')
if 'type' in numeric_cols:
    numeric_cols.remove('type')
```
[9]

```python
fig, axes = plt.subplots(4, 3, figsize=(15, 15))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    ax = axes[i]
    df[col].hist(bins=30, ax=ax, alpha=0.7, color='skyblue', edgecolor='black')
    ax.set_title(f'Распределение {col}', fontsize=12, fontweight='bold')
    ax.set_xlabel(col)
    ax.set_ylabel('Частота')
    ax.grid(True, alpha=0.3)

plt.suptitle('РАСПРЕДЕЛЕНИЕ ЧИСЛОВЫХ ПРИЗНАКОВ', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```
[10]

Рисунок 13 – Лабораторная работа №2

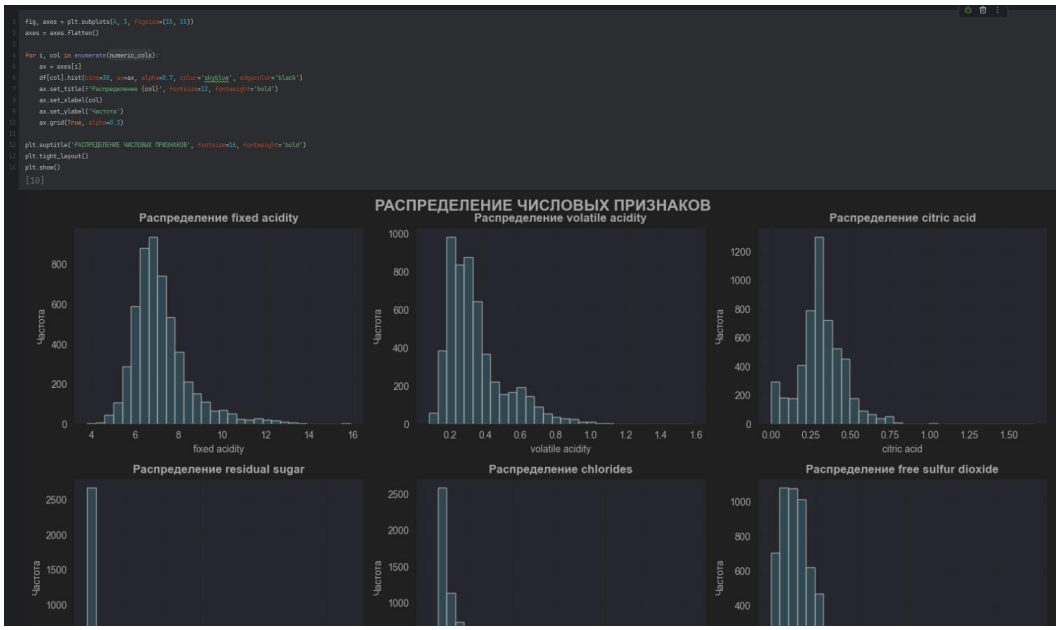


Рисунок 14 – Лабораторная работа №2

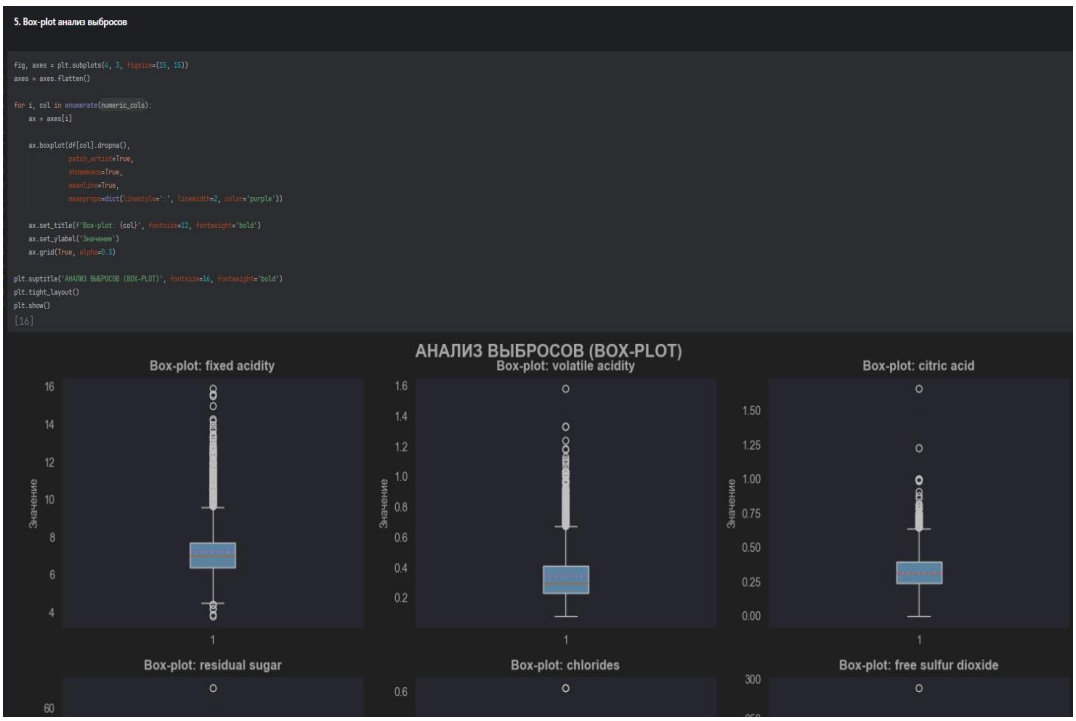


Рисунок 15 – Лабораторная работа №2

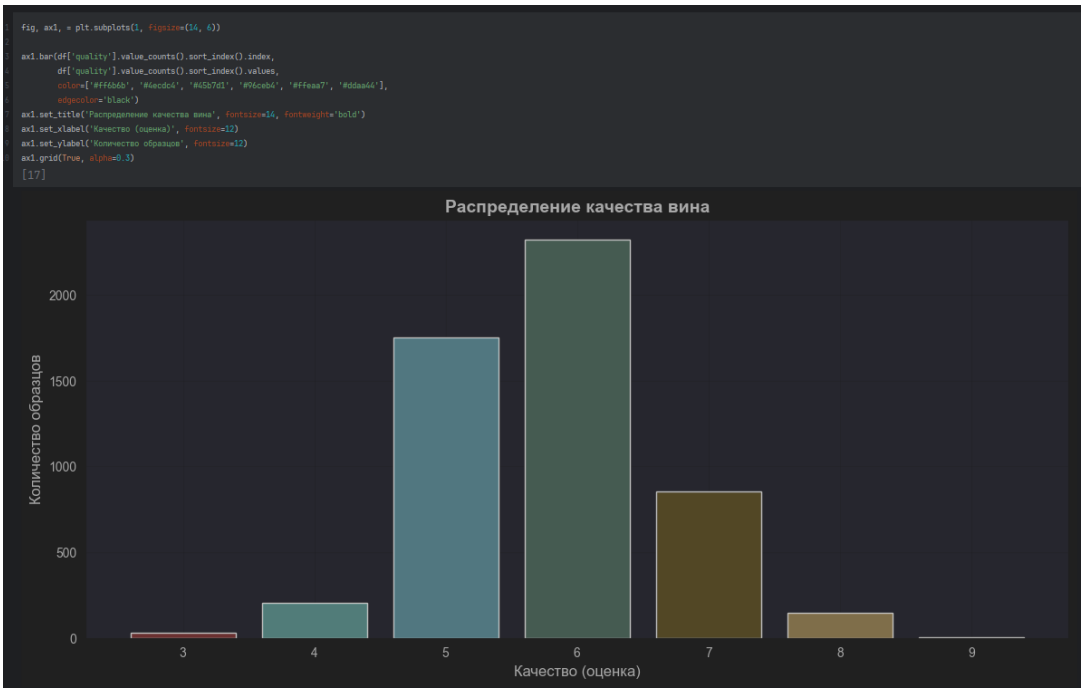


Рисунок 16 – Лабораторная работа №2

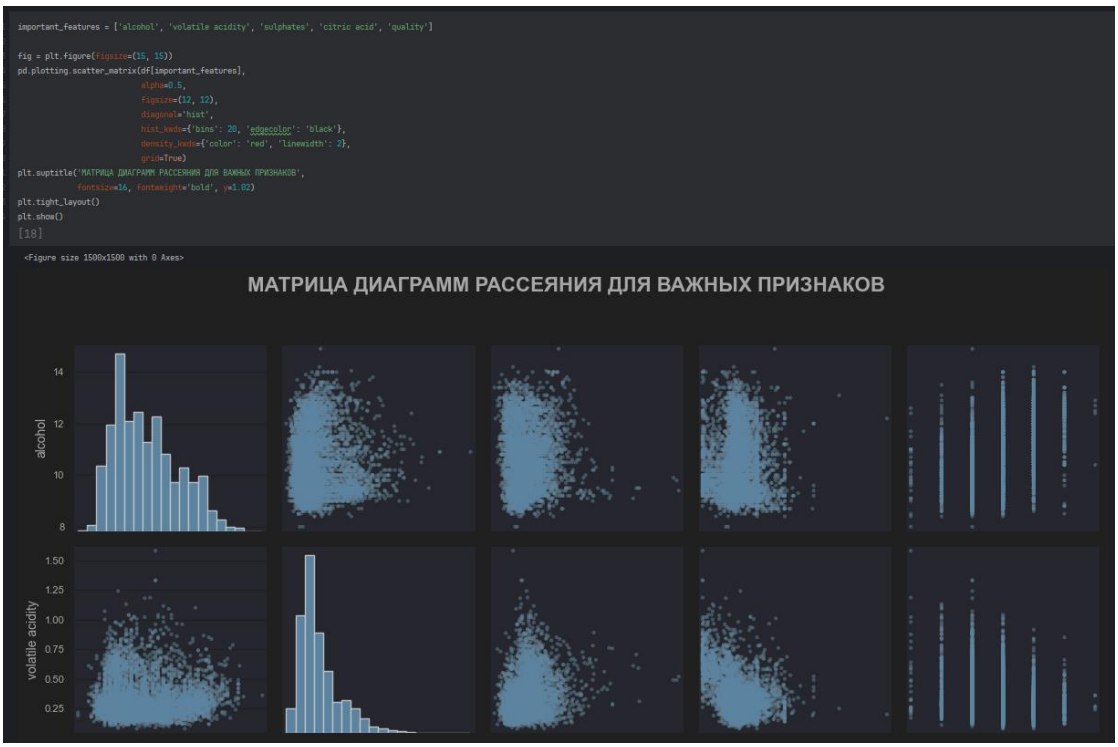


Рисунок 17 – Лабораторная работа №2

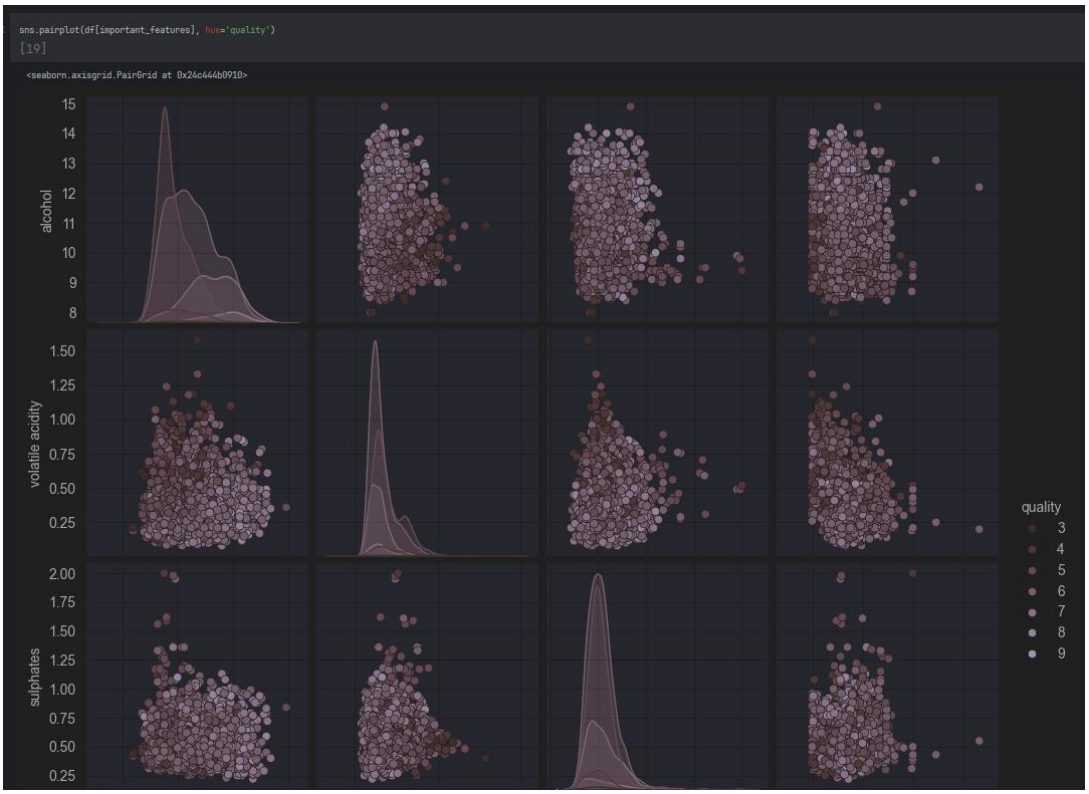


Рисунок 18 – Лабораторная работа №2

```python
fig = plt.subplots(1, figsize=(18, 7))

correlation_matrix = df[numeric_cols].corr()

sns.heatmap(correlation_matrix,
            annot=True,
            fmt='.2f',
            cmap='coolwarm',
            center=0,
            square=True,
            linewidths=0.5,
            linecolor='white',
            cbar_kws={'shrink': 0.8, 'label': 'Коэффициент корреляции'},
            annot_kws={'size': 9, 'weight': 'bold'})

plt.title('ТЕПЛОВАЯ КАРТА КОРРЕЛЯЦИЙ МЕЖДУ ПРИЗНАКАМИ ВИНА',
          fontsize=16, fontweight='bold', pad=20)
plt.xticks(rotation=45, ha='right', fontsize=11)
plt.yticks(rotation=0, fontsize=11)
plt.tight_layout()
plt.show()
```

[20]

Рисунок 19 – Лабораторная работа №2

Лабораторная работа №3. Бинарная классификация.

Весь процесс и результат лабораторной работы №3 представлены на рисунках 19-30.

Задание 3

1. Импортируйте необходимые библиотеки

```
import ...
[37]
```

2. Загрузить набор данных как датафрейм библиотеки pandas. Использовать тот же набор данных, что и в задании 2.

```
df = pd.read_csv('wine.csv')
[38]
```

3. Получить представлении о наборе данных с помощью методов shape, head, describe, info библиотеки pandas.

```
df.head(10)
[39]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	type
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	red
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	red
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	red
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	red
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	red
5	7.4	0.66	0.00	1.8	0.075	13.0	40.0	0.9978	3.51	0.56	9.4	5	red
6	7.9	0.60	0.06	1.6	0.069	15.0	59.0	0.9964	3.30	0.46	9.4	5	red
7	7.3	0.65	0.00	1.2	0.065	15.0	21.0	0.9946	3.39	0.47	10.0	7	red
8	7.8	0.58	0.02	2.0	0.073	9.0	18.0	0.9968	3.36	0.57	9.5	7	red
9	7.5	0.50	0.36	6.1	0.071	17.0	102.0	0.9978	3.35	0.80	10.5	5	red

Рисунок 19 – Лабораторная работа №3

```
df.info()
[40]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         6497 non-null   float64
 1   volatile acidity      6497 non-null   float64
 2   citric acid           6497 non-null   float64
 3   residual sugar        6497 non-null   float64
 4   chlorides             6497 non-null   float64
 5   free sulfur dioxide   6497 non-null   float64
 6   total sulfur dioxide  6497 non-null   float64
 7   density               6497 non-null   float64
 8   pH                    6497 non-null   float64
 9   sulphates             6497 non-null   float64
 10  alcohol               6497 non-null   float64
 11  quality               6497 non-null   int64
 12  type                  6497 non-null   object
dtypes: float64(11), int64(1), object(1)
memory usage: 660.0+ KB
```

```
df.describe()
[41]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000
mean	7.215307	0.339666	0.318633	5.443235	0.056034	30.525319	115.744574	0.994697	3.218501	0.531268	10.491801	5.818378
std	1.296434	0.164636	0.145318	4.757804	0.035034	17.749400	56.521855	0.002999	0.160787	0.148806	1.192712	0.873255
min	3.800000	0.080000	0.000000	0.600000	0.009000	1.000000	6.000000	0.987110	2.720000	0.220000	8.000000	3.000000
25%	6.400000	0.230000	0.250000	1.800000	0.038000	17.000000	77.000000	0.992340	3.110000	0.430000	9.500000	5.000000
50%	7.000000	0.290000	0.310000	3.000000	0.047000	29.000000	118.000000	0.994890	3.210000	0.510000	10.300000	6.000000
75%	7.700000	0.400000	0.390000	8.100000	0.065000	41.000000	156.000000	0.996990	3.320000	0.600000	11.300000	6.000000
max	15.900000	1.580000	1.660000	65.800000	0.611000	289.000000	440.000000	1.038980	4.010000	2.000000	14.900000	9.000000

```
rows, columns = df.shape
print(rows)
print(columns)
[42]
6497
13
```

Рисунок 20 – Лабораторная работа №3

```
df.isnull().sum()
[43]
```

```
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
type                    0
dtype: int64
```

```
df.duplicated().sum()
[44]
```

```
np.int64(1177)
```

```
df = df.drop_duplicates().reset_index(drop=True)
[45]
```

```
df.duplicated().sum()
[46]
```

```
np.int64(0)
```

4. Проверить сбалансированность набора данных.

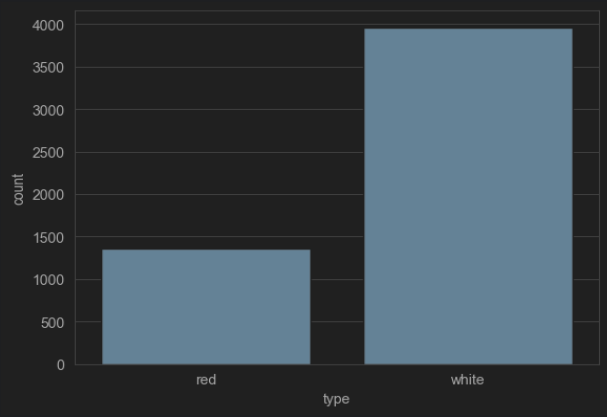
```
(unique, counts) = np.unique(df['type'], return_counts=True)
print(unique)
print(counts)
[47]
```

```
['red' 'white']
[1359 3961]
```

Рисунок 21 – Лабораторная работа №3

```
sns.set_style("whitegrid")
sns.countplot(df, x = "type")
[48]
```

```
<Axes: xlabel='type', ylabel='count'>
```



4. Произвести разведочный анализ данных по датасету – визуализировать данные с помощью библиотек matplotlib, seaborn.

```
numeric_features = df.select_dtypes(include=[np.number]).columns.tolist()
if 'quality' in numeric_features:
    numeric_features.remove('quality')

fig, axes = plt.subplots(3, 4, figsize=(16, 10))
axes = axes.flatten()

for i, col in enumerate(numeric_features):
    if i < len(axes):
        axes[i].hist(df[col], bins=30, edgecolor='black', alpha=0.7)
        axes[i].set_title(f'Распределение {col}', fontsize=10)
        axes[i].set_xlabel(col)
        axes[i].set_ylabel('Частота')

plt.tight_layout()
plt.show()
[49]
```

Рисунок 22 – Лабораторная работа №3

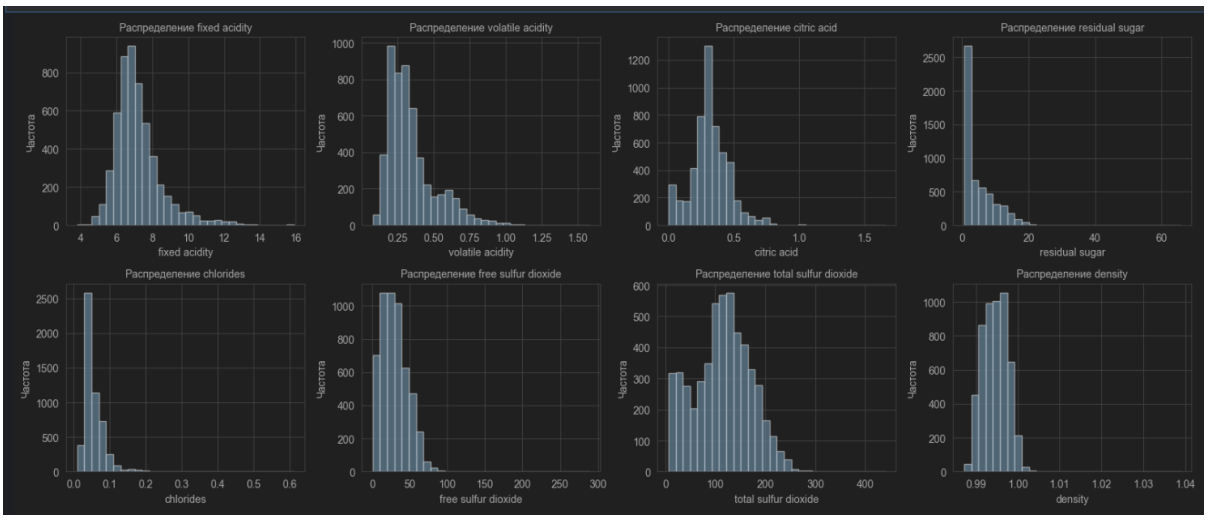


Рисунок 23 – Лабораторная работа №3

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(data=df, x='type', y='quality', ax=axes[0], legend=False)
axes[0].set_title('Распределение качества по типам вина', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Тип вина')
axes[0].set_ylabel('Качество')

quality_counts = df['quality'].value_counts().sort_index()
axes[1].bar(quality_counts.index, quality_counts.values, color='skyblue', edgecolor='black')
axes[1].set_title('Общее распределение качества вина', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Качество')
axes[1].set_ylabel('Количество')
for i, v in enumerate(quality_counts.values):
    axes[1].text(quality_counts.index[i], v + 50, str(v), ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
[50]
```

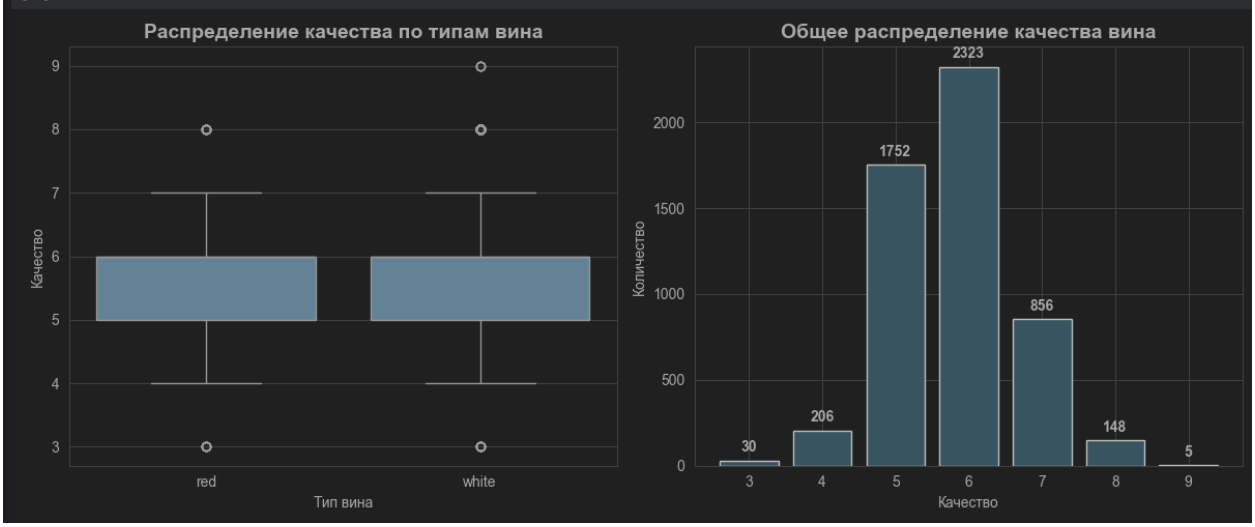


Рисунок 24 – Лабораторная работа №3

```python
df_numeric = df.select_dtypes(include=[np.number])

label_encoder = LabelEncoder()
df_numeric['type_numeric'] = label_encoder.fit_transform(df['type'])

plt.figure(figsize=(8, 8))
correlation_matrix = df_numeric.corr()
mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, mask=mask, square=True)
plt.title('Матрица корреляций признаков', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```
✓ [36] 241ms

Рисунок 25 – Лабораторная работа №3

```python
correlation_with_target = correlation_matrix['type_numeric'].abs().sort_values(ascending=False)
correlation_with_target[1:]
```
✓ [18] < 10 ms

```
total sulfur dioxide    0.694229
volatile acidity        0.645335
chlorides               0.499517
sulphates               0.490364
fixed acidity           0.486253
free sulfur dioxide     0.465326
density                 0.429377
residual sugar          0.328695
pH                      0.310919
citric acid             0.183759
quality                 0.114809
alcohol                 0.057756
Name: type_numeric, dtype: float64
```

6. На основе выполненного анализа данных произвести выводы о влиянии различных признаков на класс.

```python
top_features = correlation_with_target[1:4]
print("Наиболее значимые признаки для определения типа вина:")
top_features
```
✓ [19] < 10 ms

```
Наиболее значимые признаки для определения типа вина:
total sulfur dioxide    0.694229
volatile acidity        0.645335
chlorides               0.499517
Name: type_numeric, dtype: float64
```

7. Произвести предобработку данных:

```python
df_processed = df.copy()

low_corr_features = []
for feature, corr_value in correlation_with_target.items():
    if corr_value < 0.1 and feature != 'type_numeric':
        if feature in df.columns:
            low_corr_features.append(feature)

print(f"Удаляемые признаки (корреляция < 0.1): {low_corr_features}")
df_processed = df_processed.drop(columns=low_corr_features, errors='ignore')
```
✓ [20] < 10 ms

```
Удаляемые признаки (корреляция < 0.1): ['alcohol']
```

Рисунок 26 – Лабораторная работа №3

```python
df_processed['type_encoded'] = label_encoder.fit_transform(df_processed['type'])
class_mapping = dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))
```
✓ [21] < 10 ms

```python
scaler = StandardScaler()

numeric_cols = df_processed.select_dtypes(include=[np.number]).columns.tolist()

numeric_cols = [col for col in numeric_cols if col not in ['type_encoded', 'quality']]
df_processed[numeric_cols] = scaler.fit_transform(df_processed[numeric_cols])

print(f"\nРазмер датасета после предобработки: {df_processed.shape}")
print(f"Колонки после предобработки: {df_processed.columns.tolist()}")
```
✓ [22] < 10 ms

```
Размер датасета после предобработки: (5320, 13)
Колонки после предобработки: ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'quality', 'type', 'type_encoded']
```

8. Разбить обработанный набор данных на обучающую и тестовую выборки с помощью методов библиотеки sklearn.

```python
X = df_processed.drop(['type', 'type_encoded'], axis=1, errors='ignore')
if 'quality' in X.columns:
    X = X.drop('quality', axis=1)
y = df_processed['type_encoded']
```
✓ [23] < 10 ms

```python
(unique, counts) = np.unique(df_processed['type_encoded'], return_counts=True)
print(unique)
print(counts)
```
✓ [24] < 10 ms

```
[0 1]
[1359 3961]
```

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print(f"Размер обучающей выборки: {X_train.shape}")
print(f"Размер тестовой выборки: {X_test.shape}")
```
✓ [25] 33ms

```
Размер обучающей выборки: (3724, 10)
Размер тестовой выборки: (1596, 10)
```

Рисунок 27 – Лабораторная работа №3

9. Произвести обучение модели логистическая регрессии (LogisticRegression) библиотеки sklearn.

```python
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(class_weight='balanced')
logreg.fit(X_train, y_train)
```
✓ [26] 52ms

```
LogisticRegression
▶ Parameters
```

```python
y_train_pred = logreg.predict(X_train)
y_test_pred = logreg.predict(X_test)
```
✓ [27] < 10 ms

10. Отобразить корректность работы модели на тестовой и обучающей выборках с помощью стандартных метрик библиотеки sklearn.

```python
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_train, y_train_pred)
TN, FP, FN, TP = confusion_matrix(y_train, y_train_pred).ravel()

print("TP = ", TP)
print("FP = ", FP)
print("TN = ", TN)
print("FN = ", FN)

accuracy = (TP + TN) / (TP + FP + TN + FN)
print("Accuracy = ", accuracy)
```
✓ [28] < 10 ms

```
TP =  2749
FP =  11
TN =  940
FN =  24
Accuracy =  0.9906015037593985
```

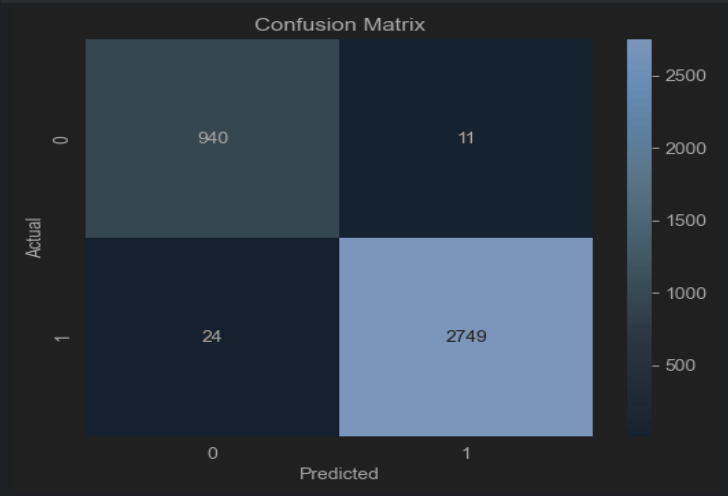
```python
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_test_pred)
print("Accuracy = ", accuracy)
```
✓ [29] < 10 ms

```
Accuracy =  0.9899749373433584
```

Рисунок 28 – Лабораторная работа №3

```
sns.heatmap(cm, square=True, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

✓ [30] 75ms




```
from sklearn.metrics import precision_score, recall_score

precision = precision_score(y_test, y_test_pred)
recall = recall_score(y_test, y_test_pred)
print("Precision = ", precision)
print("Recall = ", recall)
```

✓ [31] < 10 ms

```
Precision =  0.9949324324324325
Recall =  0.9915824915824916
```

Рисунок 29 – Лабораторная работа №3

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_test_pred)
print("F1 score = ", f1)
```

✓ [32] < 10 ms

```
F1 score =  0.9932546374367622
```

```
from sklearn.metrics import roc_curve, auc

y_score = logreg.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds_test = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)
```

✓ [33] < 10 ms

```
plt.figure(figsize=(6, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2,
         label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Случайный классификатор')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC-кривая для модели классификации типа вина', fontsize=14, fontweight='bold')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

✓ [37] 82ms

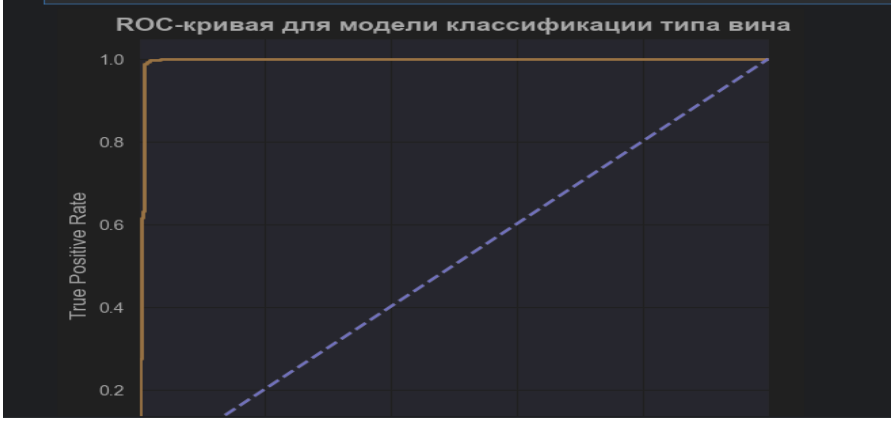


Рисунок 30 – Лабораторная работа №3

Лабораторная работа №4. Многомерная регрессия.

Весь процесс и результат лабораторной работы №4 представлены на рисунках 31-41.

Задание 4

1. Импортируйте необходимые библиотеки

```
import ...
```

✓ [2] < 10 ms

2. Загрузить набор данных как датафрейм библиотеки pandas. Использовать набор данных для задания 4.

```
df = pd.read_csv("octan.csv")
```

✓ [3] < 10 ms

3. Получить представлении о наборе данных с помощью методов shape, head, describe, info библиотеки pandas.

```
df.head(10)
```

✓ [4] 10ms

	Number	One	Material_1	Material_2	Material_3	Condition	Octane
0	1	1	55.33	1.72	54	1.66219	92.19
1	2	1	59.13	1.20	53	1.58399	92.74
2	3	1	57.39	1.42	55	1.61731	91.88
3	4	1	56.43	1.78	55	1.66228	92.80
4	5	1	55.98	1.58	54	1.63195	92.56
5	6	1	56.16	2.12	56	1.68034	92.61
6	7	1	54.85	1.17	54	1.58206	92.33
7	8	1	52.83	1.50	58	1.54998	92.22
8	9	1	54.52	0.87	57	1.56230	91.56
9	10	1	54.12	0.88	57	1.57818	92.17

Рисунок 31 – Лабораторная работа №4

```
df.info()
```

✓ [5] < 10 ms

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 82 entries, 0 to 81
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Number      82 non-null     int64
 1   One         82 non-null     int64
 2   Material_1  82 non-null     float64
 3   Material_2  82 non-null     float64
 4   Material_3  82 non-null     int64
 5   Condition   82 non-null     float64
 6   Octane      82 non-null     float64
dtypes: float64(4), int64(3)
memory usage: 4.6 KB
```

```
df.describe()
```

✓ [6] 13ms

	Number	One	Material_1	Material_2	Material_3	Condition	Octane
count	82.000000	82.0	82.00000	82.000000	82.000000	82.000000	82.000000
mean	41.500000	1.0	60.17061	1.663659	55.463415	1.626571	91.849878
std	23.815261	0.0	12.43740	1.901952	5.377583	0.219730	1.401124
min	1.000000	1.0	4.23000	0.000000	40.000000	1.199750	89.660000
25%	21.250000	1.0	55.38000	0.105000	54.000000	1.518095	90.847500
50%	41.500000	1.0	62.69500	1.280000	56.000000	1.603575	91.735000
75%	61.750000	1.0	67.78000	2.277500	59.750000	1.722818	92.467500
max	82.000000	1.0	75.54000	10.760000	64.000000	2.319090	97.610000

```
rows, columns = df.shape
print(rows)
print(columns)
```

✓ [7] < 10 ms

```
82
7
```

```
df.isnull().sum()
```

✓ [8] < 10 ms

```
Number       0
```

Рисунок 32 – Лабораторная работа №4

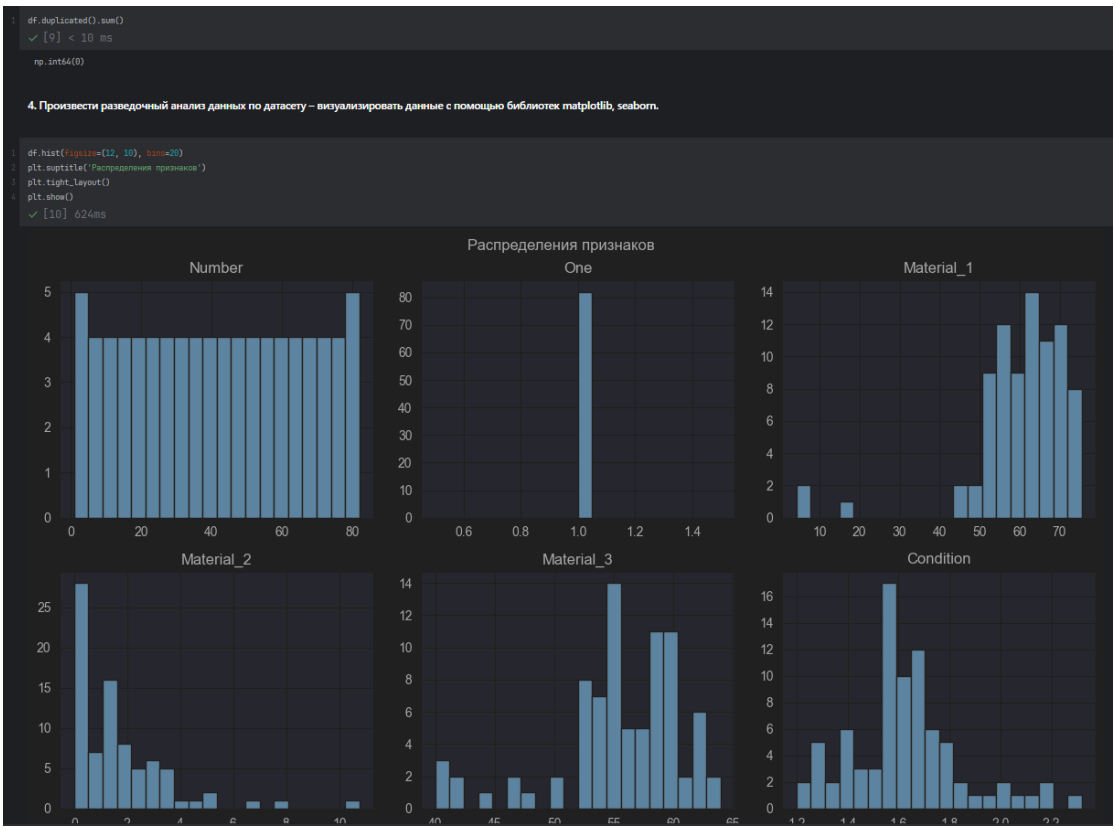


Рисунок 33 – Лабораторная работа №4

Рисунок 34 – Лабораторная работа №4

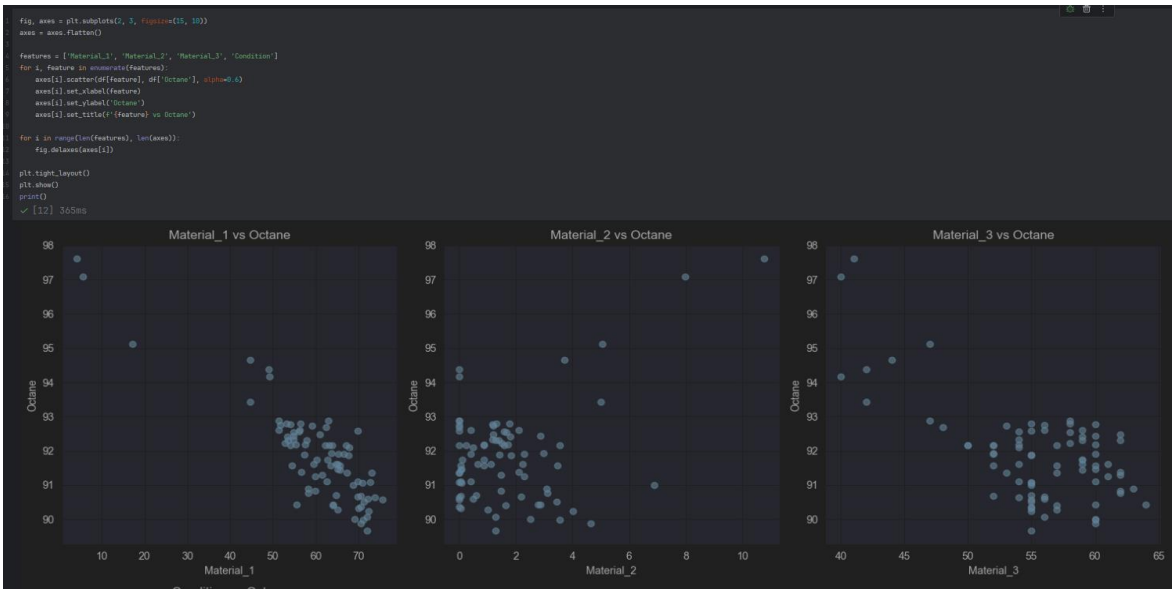
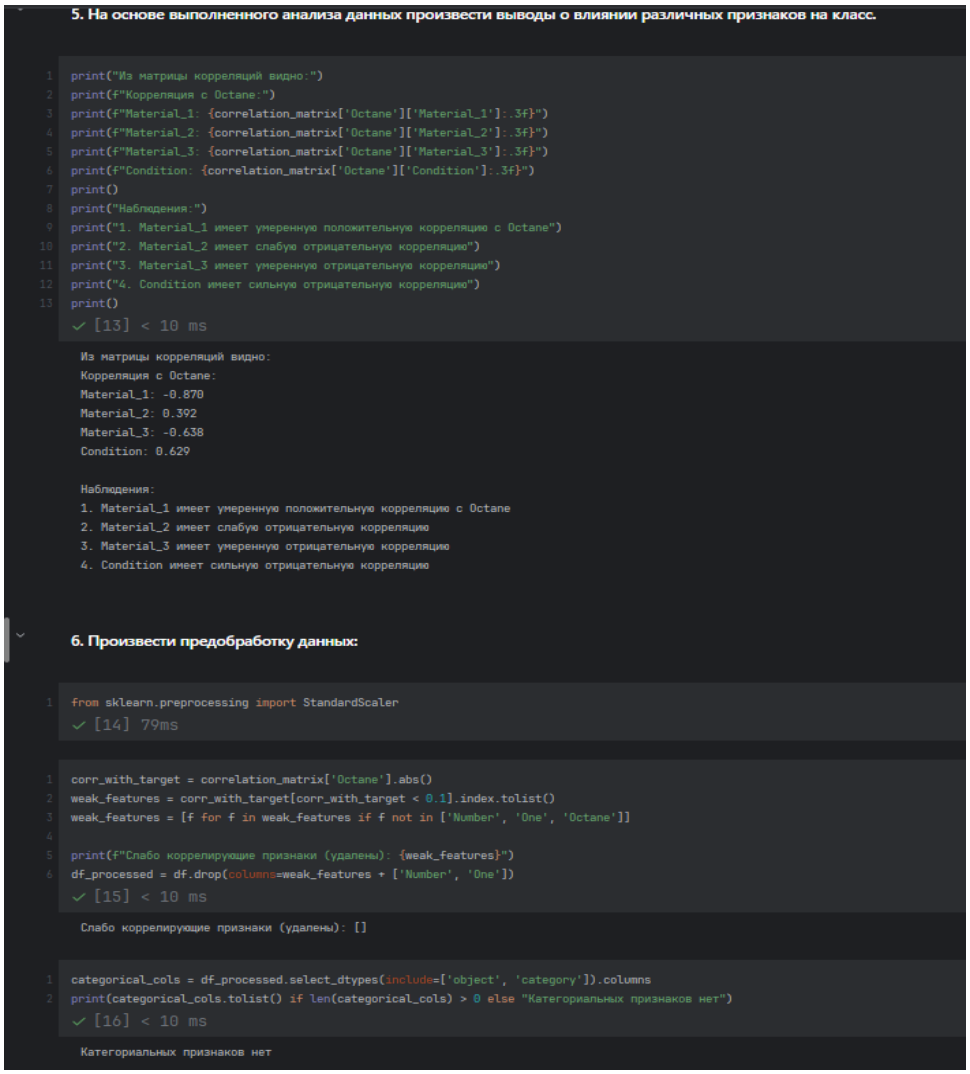


Рисунок 35 – Лабораторная работа №4



5. На основе выполненного анализа данных произвести выводы о влиянии различных признаков на класс.

```
print("Из матрицы корреляций видно:")
print(f"Корреляция с Octane:")
print(f"Material_1: {correlation_matrix['Octane']['Material_1']:.3f}")
print(f"Material_2: {correlation_matrix['Octane']['Material_2']:.3f}")
print(f"Material_3: {correlation_matrix['Octane']['Material_3']:.3f}")
print(f"Condition: {correlation_matrix['Octane']['Condition']:.3f}")
print()
print("Наблюдения:")
print("1. Material_1 имеет умеренную положительную корреляцию с Octane")
print("2. Material_2 имеет слабую отрицательную корреляцию")
print("3. Material_3 имеет умеренную отрицательную корреляцию")
print("4. Condition имеет сильную отрицательную корреляцию")
print()
[13] < 10 ms
```

```
Из матрицы корреляций видно:
Корреляция с Octane:
Material_1: -0.870
Material_2: 0.392
Material_3: -0.638
Condition: 0.629

Наблюдения:
1. Material_1 имеет умеренную положительную корреляцию с Octane
2. Material_2 имеет слабую отрицательную корреляцию
3. Material_3 имеет умеренную отрицательную корреляцию
4. Condition имеет сильную отрицательную корреляцию
```

6. Произвести предобработку данных:

```
from sklearn.preprocessing import StandardScaler
[14] 79ms
```

```
corr_with_target = correlation_matrix['Octane'].abs()
weak_features = corr_with_target[corr_with_target < 0.1].index.tolist()
weak_features = [f for f in weak_features if f not in ['Number', 'One', 'Octane']]

print(f"Слабо коррелирующие признаки (удалены): {weak_features}")
df_processed = df.drop(columns=weak_features + ['Number', 'One'])
[15] < 10 ms
```

```
Слабо коррелирующие признаки (удалены): []
```

```
categorical_cols = df_processed.select_dtypes(include=['object', 'category']).columns
print(categorical_cols.tolist() if len(categorical_cols) > 0 else "Категориальных признаков нет")
[16] < 10 ms
```

```
Категориальных признаков нет
```

Рисунок 36 – Лабораторная работа №4

```python
scaler = StandardScaler()
features_to_scale = [col for col in df_processed.columns if col != 'Octane']
df_scaled = df_processed.copy()
df_scaled[features_to_scale] = scaler.fit_transform(df_processed[features_to_scale])
```

✓ [17] < 10 ms

```python
print(df_scaled.head())
print(f"Размер: {df_scaled.shape}")
print()
```

✓ [18] < 10 ms

```
   Material_1  Material_2  Material_3  Condition  Octane
0   -0.391593    0.029805   -0.273807   0.163100   92.19
1   -0.084183   -0.245281   -0.460909  -0.194982   92.74
2   -0.224944   -0.128898   -0.086706  -0.042408   91.88
3   -0.302606    0.061546   -0.086706   0.163512   92.80
4   -0.339010   -0.044256   -0.273807   0.024629   92.56
Размер: (82, 5)
```

7. Разбить обработанный набор данных на обучающую и тестовую выборки с помощью методов библиотеки sklearn.

```python
from sklearn.model_selection import train_test_split
```

✓ [19] 29ms

```python
X = df_scaled.drop(columns=['Octane'])
y = df_scaled['Octane']
```

✓ [20] < 10 ms

```python
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
```

✓ [21] < 10 ms

8. Произвести обучение модели линейной регрессии (LinearRegression) библиотеки sklearn.

```python
from sklearn.linear_model import LinearRegression
```

✓ [22] 29ms

```python
model = LinearRegression()
model.fit(X_train, y_train)
```

✓ [23] < 10 ms

```
LinearRegression
▶ Parameters
```

Рисунок 37 – Лабораторная работа №4

9. Отобразить корректность работы модели на тестовой и обучающей выборках с помощью стандартных метрик библиотеки sklearn.

```python
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```
✓ [24] < 10 ms

```python
def evaluate_model(y_true, y_pred, dataset_name):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"\n{dataset_name}:")
    print(f"MSE: {mse:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE: {mae:.4f}")
    print(f"R²: {r2:.4f}")

    return mse, rmse, mae, r2
```
✓ [25] < 10 ms

```python
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```
✓ [26] < 10 ms

```python
train_metrics = evaluate_model(y_train, y_train_pred, "Обучающая выборка")
```
✓ [27] < 10 ms

```
Обучающая выборка:
MSE: 0.2043
RMSE: 0.4520
MAE: 0.3796
R²: 0.8893
```

```python
test_metrics = evaluate_model(y_test, y_test_pred, "Тестовая выборка")
```
✓ [28] < 10 ms

```
Тестовая выборка:
MSE: 0.1034
RMSE: 0.3216
MAE: 0.2558
R²: 0.9547
```

Рисунок 38 – Лабораторная работа №4

```python
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(y_train, y_train_pred, alpha=0.6)
axes[0].plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--', lw=2)
axes[0].set_xlabel('Фактические значения')
axes[0].set_ylabel('Предсказанные значения')
axes[0].set_title('Обучающая выборка')
axes[0].grid(True, alpha=0.3)

axes[1].scatter(y_test, y_test_pred, alpha=0.6, color='orange')
axes[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[1].set_xlabel('Фактические значения')
axes[1].set_ylabel('Предсказанные значения')
axes[1].set_title('Тестовая выборка')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```
✓ [29] 200ms

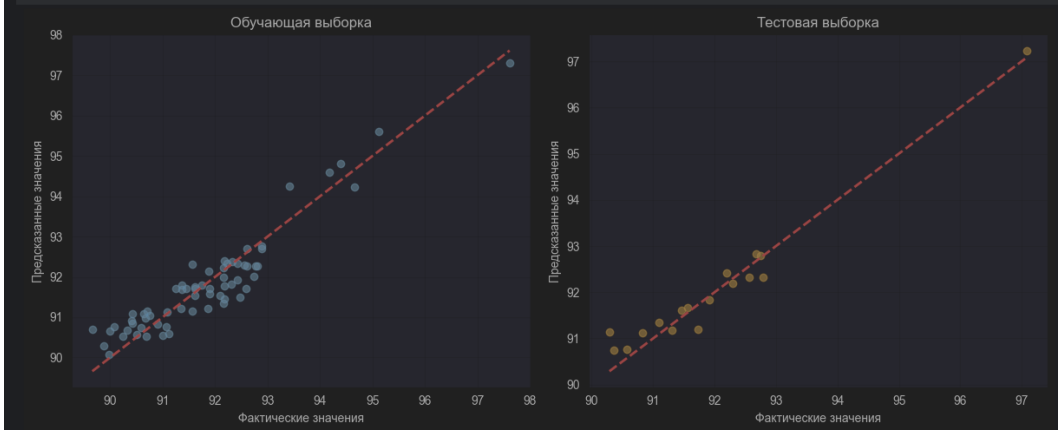


Рисунок 39 – Лабораторная работа №4

Рисунок 40 – Лабораторная работа №4

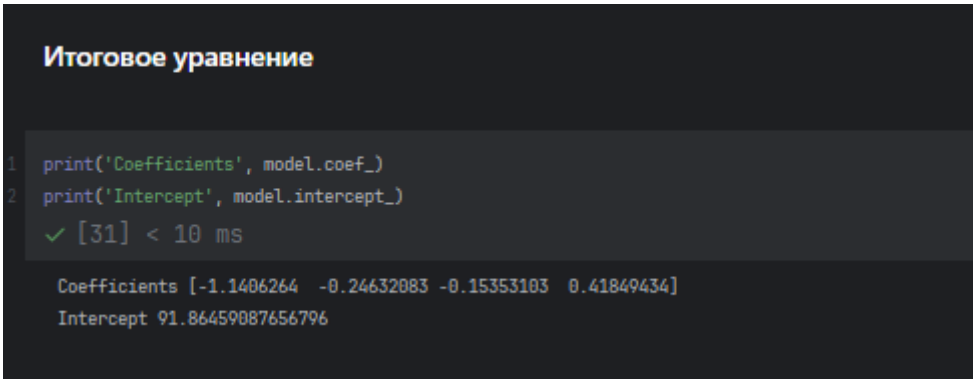


Рисунок 41 – Лабораторная работа №4

Лабораторная работа №5. Кластеризация.

Весь процесс и результат лабораторной работы №5 представлены на рисунках 42-.

Задание 5

1. Импортируйте необходимые библиотеки

```
import ...
```

✓ [1] < 10 ms

1. Загрузить набор данных как датафрейм библиотеки pandas. Использовать набор данных для задания 5.

```
df = pd.read_csv("Mall_Customers.csv")
```

✓ [2] < 10 ms

2. Получить представлении о наборе данных с помощью методов shape, head, describe, info библиотеки pandas..

```
df.head(10)
```

✓ [3] 10ms

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72

Рисунок 42 – Лабораторная работа №5

```
df.info()
```
✓ [4] < 10 ms

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Genre                   200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.describe()
```
✓ [5] 11ms

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
rows, columns = df.shape
print(rows)
print(columns)
```
✓ [6] < 10 ms

```
200
5
```

```
df.isnull().sum()
```
✓ [7] < 10 ms

```
CustomerID              0
Genre                   0
Age                     0
Annual Income (k$)      0
```

Рисунок 43 – Лабораторная работа №5

```
df.duplicated().sum()
✓ [8] < 10 ms
np.int64(0)
```

4. Произвести разведочный анализ данных по датасету – визуализировать данные с помощью библиотек matplotlib, seaborn. На основе выполненного анализа произвести выводы о распределении значений признаков.

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

axes[0].hist(df['Age'], bins=20, edgecolor='black', alpha=0.7, color='skyblue')
axes[0].set_xlabel('Возраст')
axes[0].set_ylabel('Частота')
axes[0].set_title('Распределение возраста')
axes[0].grid(True, alpha=0.3)

axes[1].hist(df['Annual Income (k$)'], bins=20, edgecolor='black', alpha=0.7, color='lightgreen')
axes[1].set_xlabel('Годовой доход (k$)')
axes[1].set_ylabel('Частота')
axes[1].set_title('Распределение годового дохода')
axes[1].grid(True, alpha=0.3)

axes[2].hist(df['Spending Score (1-100)'], bins=20, edgecolor='black', alpha=0.7, color='salmon')
axes[2].set_xlabel('Spending Score')
axes[2].set_ylabel('Частота')
axes[2].set_title('Распределение Spending Score')
axes[2].grid(True, alpha=0.3)

gender_counts = df['Genre'].value_counts()
axes[3].bar(gender_counts.index, gender_counts.values, color=['pink', 'lightblue'])
axes[3].set_xlabel('Пол')
axes[3].set_ylabel('Количество')
axes[3].set_title('Распределение по полу')
axes[3].grid(True, alpha=0.3)

scatter = axes[4].scatter(df['Annual Income (k$)'], df['Spending Score (1-100)'],
                          c=df['Age'], cmap='viridis', alpha=0.6)
axes[4].set_xlabel('Годовой доход (k$)')
axes[4].set_ylabel('Spending Score')
axes[4].set_title('Доход vs Spending Score (цвет - возраст)')
axes[4].grid(True, alpha=0.3)
plt.colorbar(scatter, ax=axes[4], label='Возраст')

axes[5].scatter(df['Age'], df['Spending Score (1-100)'],
                c=df['Annual Income (k$)'], cmap='plasma', alpha=0.6)
axes[5].set_xlabel('Возраст')
axes[5].set_ylabel('Spending Score')
axes[5].set_title('Возраст vs Spending Score (цвет - доход)')
axes[5].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Рисунок 44 – Лабораторная работа №5

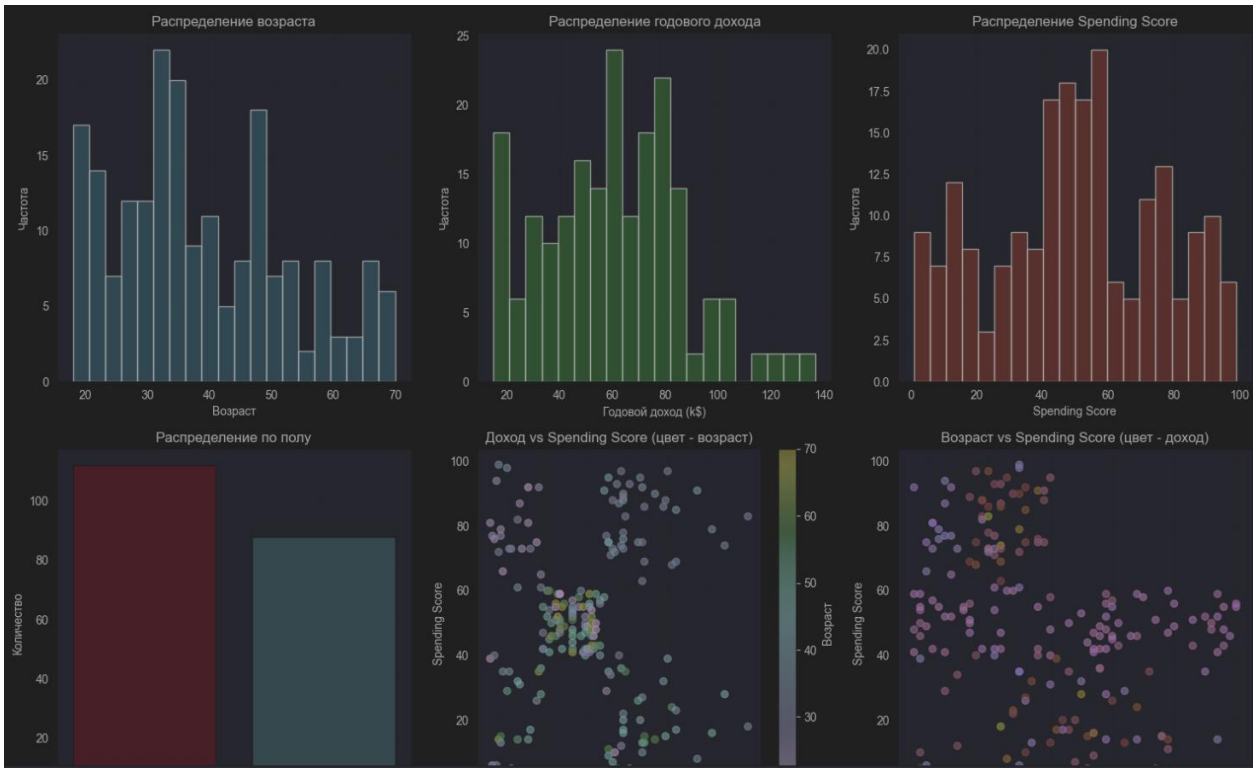


Рисунок 45 – Лабораторная работа №5

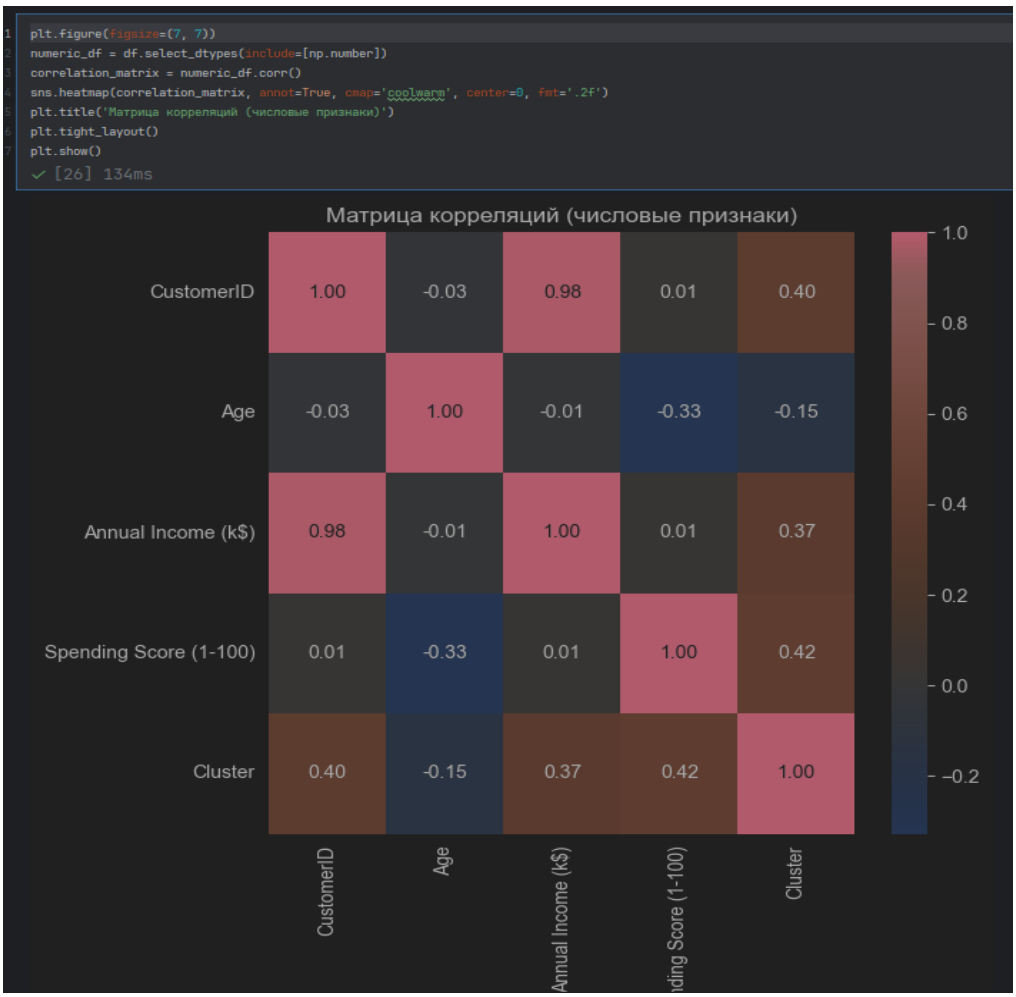


Рисунок 46 – Лабораторная работа №5

Рисунок 47 – Лабораторная работа №5

5. Произвести предобработку данных:

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```
✓ [12] 80ms

```
df_processed = df.copy()

encoder = LabelEncoder()
df_processed['Genre'] = encoder.fit_transform(df_processed['Genre'])
print(f"Кодирование признака 'Genre': {dict(zip(encoder.classes_, encoder.transform(encoder.classes_)))}")
```
✓ [13] < 10 ms

```
Кодирование признака 'Genre': {'Female': np.int64(0), 'Male': np.int64(1)}
```

```
df_processed = df_processed.drop(columns=['CustomerID'])
```
✓ [14] < 10 ms

```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df_processed)
df_scaled = pd.DataFrame(scaled_features, columns=df_processed.columns)

df_scaled.head(10)
```
✓ [15] < 10 ms

	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1.128152	-1.424569	-1.738999	-0.434801
1	1.128152	-1.281035	-1.738999	1.195704
2	-0.886405	-1.352802	-1.700830	-1.715913
3	-0.886405	-1.137502	-1.700830	1.040418
4	-0.886405	-0.563369	-1.662660	-0.395980
5	-0.886405	-1.209269	-1.662660	1.001596
6	-0.886405	-0.276302	-1.624491	-1.715913
7	-0.886405	-1.137502	-1.624491	1.700384
8	1.128152	1.804932	-1.586321	-1.832378
9	-0.886405	-0.635135	-1.586321	0.846310

Рисунок 48 – Лабораторная работа №5

6. Произвести обучение модели библиотеки sklearn, реализующей метод k-средних (k-means clustering).

```
from sklearn.cluster import KMeans
```
✓ [16] 114ms

```
inertia = []
k_range = range(1, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df_scaled)
    inertia.append(kmeans.inertia_)
```
✓ [17] 1s 700ms

```
plt.figure(figsize=(6, 6))
plt.plot(k_range, inertia, 'bo-')
plt.xlabel('Количество кластеров (k)')
plt.ylabel('Inertia (сумма квадратов расстояний)')
plt.title('Метод локтя для определения оптимального k')
plt.xticks(k_range)
plt.grid(True, alpha=0.3)
plt.show()
```
✓ [18] 98ms

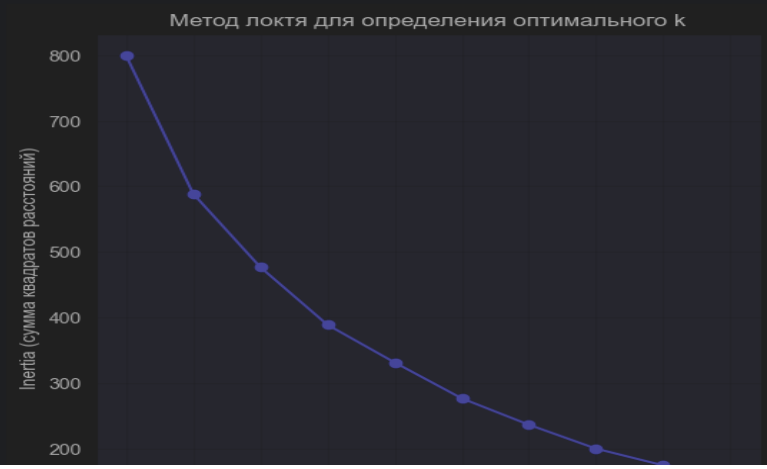


Рисунок 49– Лабораторная работа №5

```
optimal_k = 4
```
✓ [19] < 10 ms

```
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_init=4).fit(df_scaled)

clusters = kmeans.fit_predict(df_scaled)
df['Cluster'] = clusters
df_scaled['Cluster'] = clusters
```
✓ [20] 19ms

7. Произвести снижение размерности набора данных с помощью метода PCA.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_result = pca.fit_transform(df_scaled.drop(columns=['Cluster']))
df_pca = pd.DataFrame(pca_result, columns=['PC1', 'PC2'])
df_pca['Cluster'] = clusters
```
✓ [21] < 10 ms

```
plt.figure(figsize=(10, 8))
scatter = plt.scatter(df_pca['PC1'], df_pca['PC2'],
                      c=clusters, cmap='tab10', alpha=0.7, s=50)
plt.xlabel('Первая главная компонента (PC1)')
plt.ylabel('Вторая главная компонента (PC2)')
plt.title('Кластеризация клиентов (PCA визуализация)')
plt.colorbar(scatter, label='Кластер')
plt.grid(True, alpha=0.3)
plt.show()
```
✓ [22] 129ms

Рисунок 50 – Лабораторная работа №5

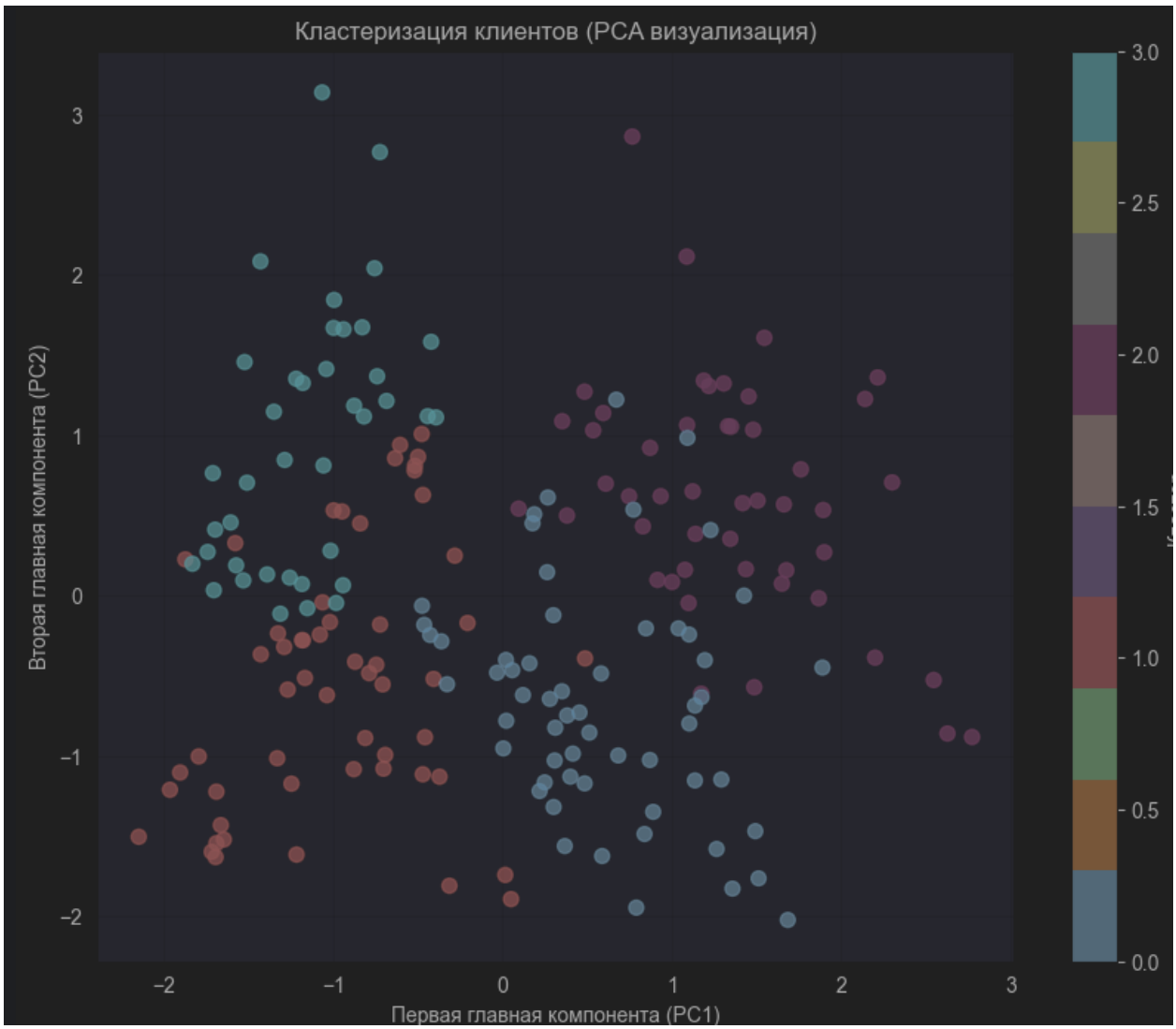


Рисунок 51 – Лабораторная работа №5

```python
centroids_scaled = kmeans.cluster_centers_
centroids_pca = pca.transform(centroids_scaled)

plt.figure(figsize=(8, 8))

colors = plt.cm.tab10(np.arange(10) / 10)

for i in range(10):
    cluster_mask = clusters == i
    plt.scatter(pca_result[cluster_mask, 0], pca_result[cluster_mask, 1],
                s=60, alpha=0.6, color=colors[i],
                label=f'Кластер {i} ({np.sum(cluster_mask)} клиентов)',
                edgecolors='white', linewidth=0.5)

centroid_scatter = plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1],
                               s=500, color='gold',
                               edgecolor='black', linewidth=2,
                               label='Центроиды', zorder=5)
```

✓ [27] 137ms

```
C:\Users\Egor\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature na
  warnings.warn(
```

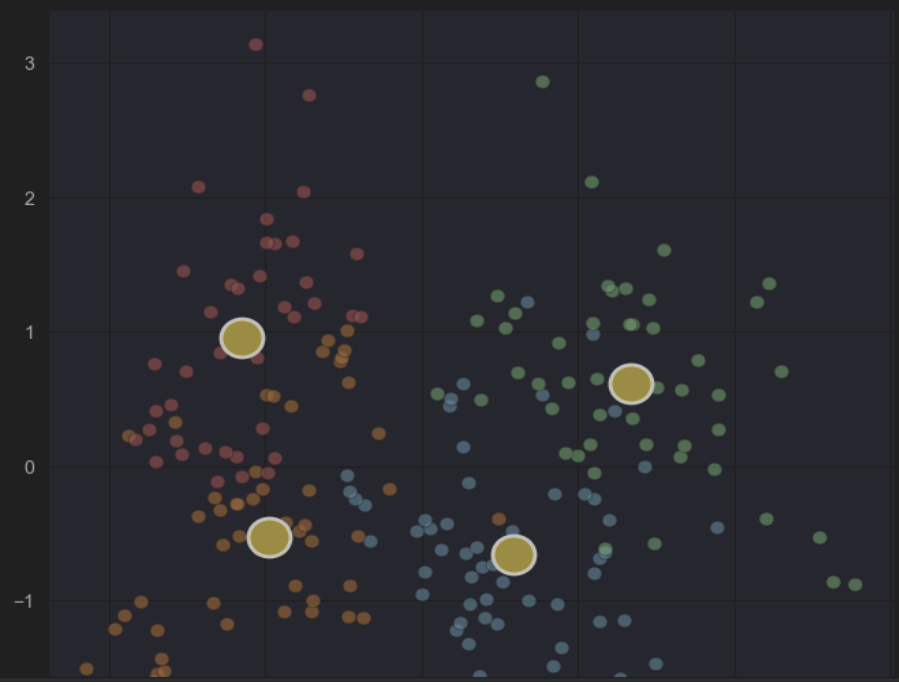


Рисунок 52 – Лабораторная работа №5

8. Произвести разведочный анализ данных по кластерам для оценки качества обучения модели.

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

cluster_counts = df['Cluster'].value_counts().sort_index()
axes[0, 0].bar(cluster_counts.index, cluster_counts.values, color=plt.cm.tab10(range(optimal_k)))
axes[0, 0].set_xlabel('Кластер')
axes[0, 0].set_ylabel('Количество клиентов')
axes[0, 0].set_title('Размеры кластеров')
axes[0, 0].set_xticks(range(optimal_k))

age_means = df.groupby('Cluster')['Age'].mean()
axes[0, 1].bar(age_means.index, age_means.values, color=plt.cm.tab10(range(optimal_k)))
axes[0, 1].set_xlabel('Кластер')
axes[0, 1].set_ylabel('Средний возраст')
axes[0, 1].set_title('Средний возраст по кластерам')
axes[0, 1].set_xticks(range(optimal_k))

income_means = df.groupby('Cluster')['Annual Income (k$)'].mean()
axes[1, 0].bar(income_means.index, income_means.values, color=plt.cm.tab10(range(optimal_k)))
axes[1, 0].set_xlabel('Кластер')
axes[1, 0].set_ylabel('Средний доход (k$)')
axes[1, 0].set_title('Средний доход по кластерам')
axes[1, 0].set_xticks(range(optimal_k))

score_means = df.groupby('Cluster')['Spending Score (1-100)'].mean()
axes[1, 1].bar(score_means.index, score_means.values, color=plt.cm.tab10(range(optimal_k)))
axes[1, 1].set_xlabel('Кластер')
axes[1, 1].set_ylabel('Средний Spending Score')
axes[1, 1].set_title('Средний Spending Score по кластерам')
axes[1, 1].set_xticks(range(optimal_k))

plt.tight_layout()
plt.show()
```

✓ [24] 283ms

Рисунок 53 – Лабораторная работа №5

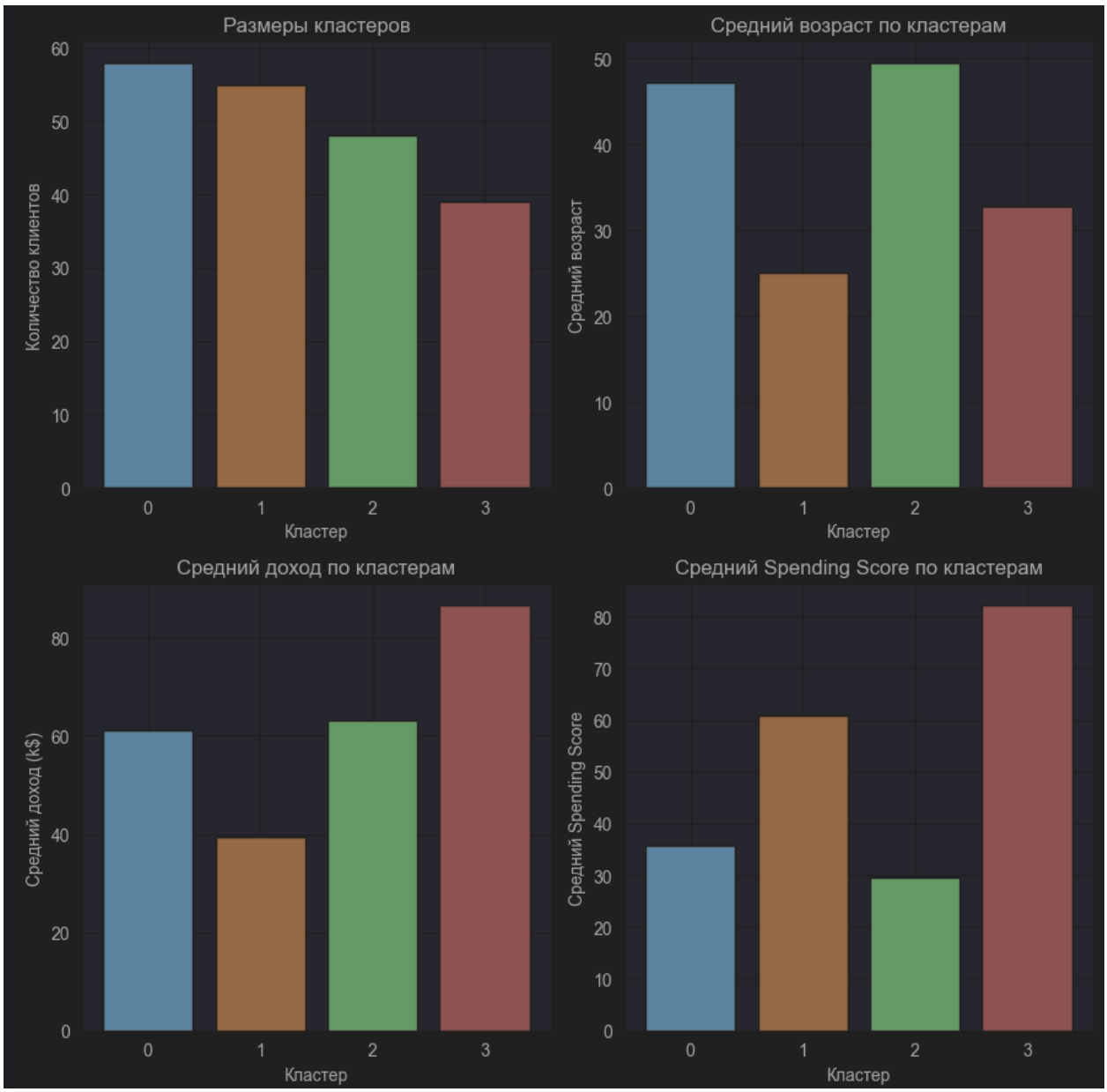


Рисунок 54 – Лабораторная работа №5

```python
from sklearn.metrics import calinski_harabasz_score, davies_bouldin_score, silhouette_score

silhouette_avg = silhouette_score(df_scaled, clusters)
print("ОЦЕНКА КАЧЕСТВА КЛАСТЕРИЗАЦИИ")
print(f"\n1. Silhouette Score: {silhouette_avg:.4f}")
print("   Интерпретация:")
if silhouette_avg > 0.7:
    print("    Отличная кластеризация")
elif silhouette_avg > 0.5:
    print("   ✓Хорошая кластеризация")
elif silhouette_avg > 0.25:
    print("   ○Умеренная кластеризация")
else:
    print(" Низкое качество")

ch_score = calinski_harabasz_score(df_scaled, clusters)
print(f"\n2. Calinski-Harabasz Index: {ch_score:.2f}")
print("   (Отношение дисперсии между кластерами к дисперсии внутри кластеров)")
print("   Чем выше значение, тем лучше разделение кластеров")

db_score = davies_bouldin_score(df_scaled, clusters)
print(f"\n3. Davies-Bouldin Index: {db_score:.4f}")
print("   (Среднее сходство каждого кластера с наиболее похожим кластером)")
print("   0 - лучший результат, чем меньше значение, тем лучше")

inertia = kmeans.inertia_
print(f"\n4. Inertia (WCSS): {inertia:.2f}")
print("   (Сумма квадратов расстояний точек до центроидов их кластеров)")
print("   Чем меньше значение, тем более компактны кластеры")
```

✓ [25] 11ms

```
ОЦЕНКА КАЧЕСТВА КЛАСТЕРИЗАЦИИ

1. Silhouette Score: 0.3956
   Интерпретация:
   ○Умеренная кластеризация

2. Calinski-Harabasz Index: 108.69
   (Отношение дисперсии между кластерами к дисперсии внутри кластеров)
   Чем выше значение, тем лучше разделение кластеров

3. Davies-Bouldin Index: 1.0276
   (Среднее сходство каждого кластера с наиболее похожим кластером)
   0 - лучший результат, чем меньше значение, тем лучше

4. Inertia (WCSS): 390.02
   (Сумма квадратов расстояний точек до центроидов их кластеров)
   Чем меньше значение, тем более компактны кластеры
```

Рисунок 55 – Лабораторная работа №5

Лабораторная работа №6. Решение задачи классификации методом решающих деревьев.

Весь процесс и результат лабораторной работы №6 представлены на рисунках 56-.

Задание 5

1. Импортируйте необходимые библиотеки

```
import pandas as pd
[1]
```

2. Загрузить набор данных как датафрейм библиотеки pandas. Использовать набор данных для задания 3 (бинарная классификация).

```
df = pd.read_csv("wine.csv")
[2]
```

3. Получить представлении о наборе данных с помощью методов shape, head, describe, info библиотеки pandas.

```
df.head(10)
[3]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	type
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	red
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	red
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	red
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	red
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	red
5	7.4	0.66	0.00	1.8	0.075	13.0	40.0	0.9978	3.51	0.56	9.4	5	red
6	7.9	0.60	0.06	1.6	0.069	15.0	59.0	0.9964	3.30	0.46	9.4	5	red
7	7.3	0.65	0.00	1.2	0.065	15.0	21.0	0.9946	3.39	0.47	10.0	7	red
8	7.8	0.58	0.02	2.0	0.073	9.0	18.0	0.9968	3.36	0.57	9.5	7	red
9	7.5	0.50	0.36	6.1	0.071	17.0	102.0	0.9978	3.35	0.80	10.5	5	red

Рисунок 56 – Лабораторная работа №6

```
df.info()
[4]

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         6497 non-null   float64
 1   volatile acidity      6497 non-null   float64
 2   citric acid           6497 non-null   float64
 3   residual sugar        6497 non-null   float64
 4   chlorides             6497 non-null   float64
 5   free sulfur dioxide   6497 non-null   float64
 6   total sulfur dioxide  6497 non-null   float64
 7   density               6497 non-null   float64
 8   pH                    6497 non-null   float64
 9   sulphates             6497 non-null   float64
 10  alcohol               6497 non-null   float64
 11  quality               6497 non-null   int64
 12  type                  6497 non-null   object
dtypes: float64(11), int64(1), object(1)
memory usage: 660.0+ KB
```

```
df.describe()
[5]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000
mean	7.215307	0.339666	0.318633	5.443235	0.056034	30.525319	115.744574	0.994697	3.218501	0.531268	10.491801	5.818378
std	1.296434	0.164636	0.145318	4.757804	0.035034	17.749400	56.521855	0.002999	0.160787	0.148806	1.192712	0.873255
min	3.800000	0.080000	0.000000	0.600000	0.009000	1.000000	6.000000	0.987110	2.720000	0.220000	8.000000	3.000000
25%	6.400000	0.230000	0.250000	1.800000	0.038000	17.000000	77.000000	0.992340	3.110000	0.430000	9.500000	5.000000
50%	7.000000	0.290000	0.310000	3.000000	0.047000	29.000000	118.000000	0.994890	3.210000	0.510000	10.300000	6.000000
75%	7.700000	0.400000	0.390000	8.100000	0.065000	41.000000	156.000000	0.996990	3.320000	0.600000	11.300000	6.000000
max	15.900000	1.580000	1.660000	65.800000	0.611000	289.000000	440.000000	1.038980	4.010000	2.000000	14.900000	9.000000

Рисунок 57 – Лабораторная работа №6

```
rows, columns = df.shape
print(rows)
print(columns)
[6]

6497
13
```

3. Произвести предобработку данных:

```
df.isnull().sum()
[7]

fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
type                    0
dtype: int64
```

```
df.duplicated().sum()
[8]

np.int64(1177)
```

```
df = df.drop_duplicates().reset_index(drop=True)
[9]
```

```
df.duplicated().sum()
[10]

np.int64(0)
```

```
df['type'].value_counts()
[11]

type
white    3961
red      1359
Name: count, dtype: int64
```

Рисунок 58 – Лабораторная работа №6

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
[12]
```

```
df_processed = df.copy()

le = LabelEncoder()
df_processed['type'] = le.fit_transform(df['type'])
print(f"\nКодировка: {list(le.classes_)} -> {list(le.transform(le.classes_))}")
[33]
```

```
Кодировка: ['red', 'white'] -> [np.int64(0), np.int64(1)]
```

```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df_processed.drop('type', axis=1))
df_scaled = pd.DataFrame(scaled_features, columns=df_processed.drop('type', axis=1).columns)

df_scaled.head(10)
[14]
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	0.140064	2.115349	-2.164515	-0.699699	0.523880	-1.069272	-1.411143	1.100996	1.779304	0.177941	-0.969152	-0.904497
1	0.443199	3.185297	-2.164515	-0.544135	1.120736	-0.282905	-0.829839	0.763753	-0.153797	0.979389	-0.631833	-0.904497
2	0.443199	2.471998	-1.892672	-0.610806	0.957957	-0.844596	-1.058837	0.831202	0.220351	0.779027	-0.631833	-0.904497
3	3.019841	-0.381197	1.641293	-0.699699	0.496751	-0.732258	-0.953146	1.168444	-0.403229	0.311515	-0.631833	0.232268
4	0.140064	1.877583	-2.164515	-0.721923	0.496751	-0.956934	-1.305451	1.100996	1.779304	0.177941	-0.969152	-0.904497
5	0.518982	1.520933	-1.756750	-0.766370	0.333972	-0.844596	-0.970761	0.628856	0.469784	-0.489933	-0.969152	-0.904497
6	0.064281	1.818141	-2.164515	-0.855263	0.225453	-0.844596	-1.640142	0.021820	1.031007	-0.423146	-0.463174	1.369032
7	0.443199	1.402050	-2.028594	-0.677476	0.442491	-1.181611	-1.692988	0.763753	0.843932	0.244728	-0.884822	1.369032
8	0.215848	0.926517	0.282076	0.233684	0.388231	-0.732258	-0.213304	1.100996	0.781574	1.780838	-0.041525	-0.904497
9	-0.390421	1.402050	-1.620828	-0.721923	1.093606	-0.844596	-0.865069	0.460235	0.345068	0.044366	-1.137811	-0.904497

5. Произвести создание и обучение модели библиотеки sklearn, реализующей Random Forest (случайный лес).

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix
[15]
```

Рисунок 59 – Лабораторная работа №6

```
X = df_scaled
y = df_processed['type']
[16]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
[17]
```

```
rf = RandomForestClassifier(random_state=42, n_jobs=-1, n_estimators=100)
rf.fit(X_train, y_train)
[18]
```

```
RandomForestClassifier
▶ Parameters
```

```
y_pred = rf.predict(X_test)
[19]
```

```
print("\nОценка базовой модели:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))
[20]
```

```
Оценка базовой модели:
Accuracy: 0.9944

Classification Report:
              precision    recall  f1-score   support

         red       0.99      0.99      0.99       272
       white       0.99      1.00      1.00       792

    accuracy                           0.99      1064
   macro avg       0.99      0.99      0.99      1064
weighted avg       0.99      0.99      0.99      1064
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
[21]
```

```
Confusion Matrix:
[[268   4]
 [  2 790]]
```

Рисунок 60 – Лабораторная работа №6

6. Подобрать гиперпараметры для улучшения модели.

```
from sklearn.model_selection import GridSearchCV
```
[22]

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}
```
[23]

{} Code

```
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    param_grid,
    cv=3,
    n_jobs=-1,
    verbose=1,
    scoring='accuracy'
)
```
[24]

```
grid_search.fit(X_train, y_train)
```
[25]

```
Fitting 3 folds for each of 162 candidates, totalling 486 fits
```

```
▸ GridSearchCV
▸ best_estimator_: RandomForestClassifier
  ▸ RandomForestClassifier
```

```
print(f"\nЛучшие параметры: {grid_search.best_params_}")
print(f"Лучшая точность (кросс-валидация): {grid_search.best_score_:.4f}")
```
[26]

```
Лучшие параметры: {'max_depth': 20, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Лучшая точность (кросс-валидация): 0.9939
```

```
best_rf = grid_search.best_estimator_
y_pred_best = best_rf.predict(X_test)
```
[27]

Рисунок 61 – Лабораторная работа №6

7. Оценить корректность построенной модели.

```
print(f"\nAccuracy улучшенной модели: {accuracy_score(y_test, y_pred_best):.4f}")
print("\nClassification Report улучшенной модели:")
print(classification_report(y_test, y_pred_best, target_names=le.classes_))
```
[28]

```
Accuracy улучшенной модели: 0.9944

Classification Report улучшенной модели:
              precision    recall  f1-score   support

         red       0.99      0.99      0.99       272
       white       0.99      1.00      1.00       792

    accuracy                           0.99      1064
   macro avg       0.99      0.99      0.99      1064
weighted avg       0.99      0.99      0.99      1064
```

```
print("\nConfusion Matrix улучшенной модели:")
print(confusion_matrix(y_test, y_pred_best))
```
[29]

```
Confusion Matrix улучшенной модели:
[[268   4]
 [  2 790]]
```

```
import matplotlib.pyplot as plt
```
[30]

```
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': best_rf.feature_importances_
}).sort_values('importance', ascending=False)

print(feature_importance)
```
[31]

```
                 feature  importance
6   total sulfur dioxide    0.287624
4              chlorides    0.256175
1       volatile acidity    0.127987
7                density    0.068514
5    free sulfur dioxide    0.059693
9              sulphates    0.051964
0          fixed acidity    0.050492
3         residual sugar    0.046618
8                     pH    0.023336
2            citric acid    0.014065
```

Рисунок 62 – Лабораторная работа №6

```
plt.figure(figsize=(10, 6))
plt.barh(feature_importance['feature'], feature_importance['importance'])
plt.xlabel('Важность признака')
plt.title('Важность признаков в модели Random Forest')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
[32]
```

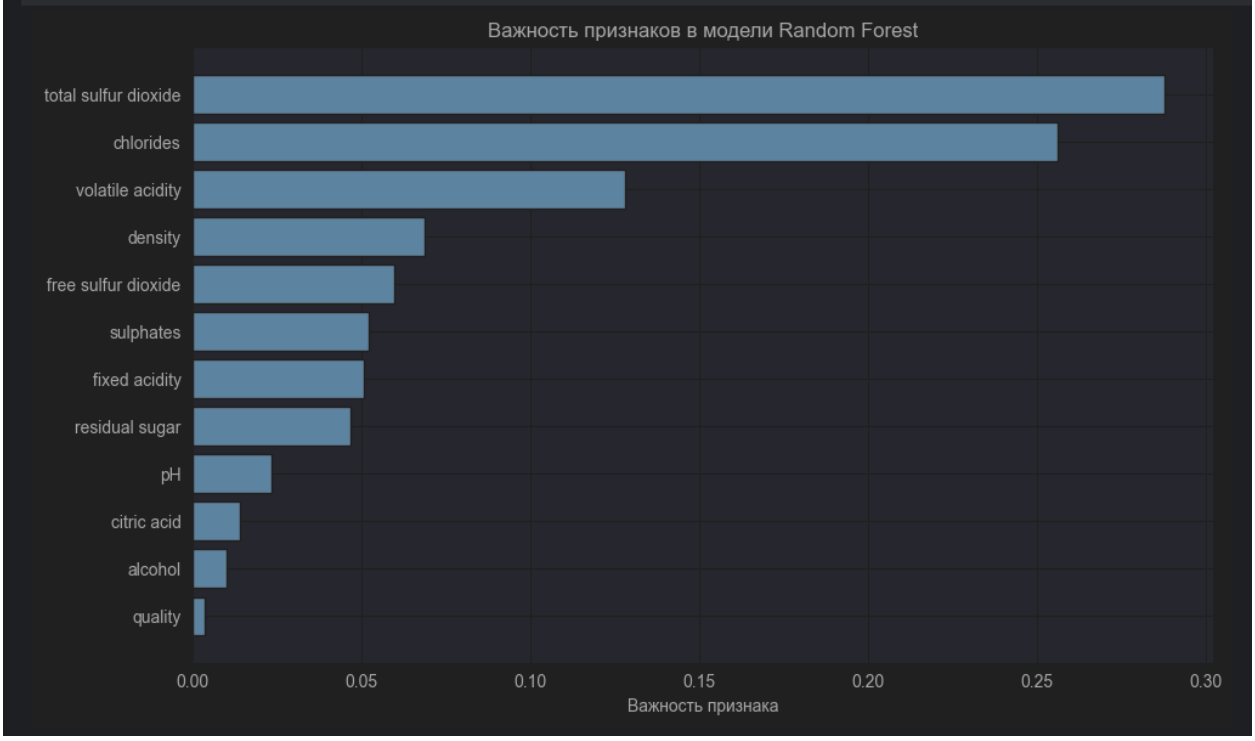


Рисунок 63 – Лабораторная работа №6

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторных работ были изучены методы обработки, анализа и визуализации данных, а также применения машинного обучения для решения практических задач. Работа с pandas позволила эффективно загружать, фильтровать и анализировать данные, выявлять зависимости между характеристиками объектов. Визуальный анализ показал, как графики и диаграммы помогают быстро обнаруживать закономерности и аномалии. Использование методов бинарной классификации, многомерной регрессии, кластеризации и алгоритмов решающих деревьев продемонстрировало возможности прогнозирования целевых показателей, сегментации объектов и оценки факторов, влияющих на результаты. Выполнение лабораторных работ позволило сформировать навыки анализа, моделирования и визуализации данных, необходимые для решения реальных задач в области анализа данных и машинного обучения.